CADT
Cambodia Academy of Digital Technology

CADT IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Khmer Sign Language Translation System

Course Project Report

Supervised by Dr. Ly Rottana

Capstone Project 01 Course Final Project - Year 3
Data Science Specialization - Computer Science Department

December 21st, 2025

Capstone Project Team 15

Member:
Vann Vat, Chim Panhaprasith, Chhi Hangcheav, Toek Hengsreng,
Mony Meakputsoktheara, Phal Sovandy

Khmer Sign Language Translation System:

A Web-Based Real-Time Gesture Recognition Application

Vann Vat
SID: IDTB100127, Email: Vat.vann@student.cadt.edu.kh

Chim Panhaprasith
SID: IDTB100224, Email: Panhaprasith.chim@student.cadt.edu.kh

Chhi Hangcheav
SID: IDTB100253, Email: Hangcheav.chhi@student.cadt.edu.kh

Toek Hengsreng
SID: IDTB100139, Email: Hengsreng.toek@student.cadt.edu.kh

Mony Meakputsoktheara
SID: IDTB100219, Email: Meakputsoktheara.mony@student.cadt.edu.kh

Phal Sovandy
SID: IDTB100173, Email: Sovandy.phal@student.cadt.edu.kh

Cambodia Academy of Digital Technology (CADT)
Department of Computer Science - Data Science Specialization
Capstone Project 01
Supervised by Dr. Ly Rottana
December 21st, 2025

Acknowledgements

We would like to express our deepest appreciation to our project advisor for his invaluable guidance, insightful feedback, and continuous supervision throughout the development of this project. His expertise, constructive criticism, and academic support were instrumental in ensuring the quality and successful completion of this work.

We also wish to acknowledge the dedication, cooperation, and collective effort of all team members. The completion of this project was made possible through effective collaboration, shared responsibility, and consistent commitment from each member, whose individual contributions were vital to the overall outcome.

Lastly, we extend our sincere gratitude to our families and friends for their patience, encouragement, and unwavering moral support throughout the duration of this project.

Abstract

ប្រព័ន្ធបកប្រែភាសាសញ្ញាខ្មែរ *(Khmer Sign Language Translation System)* គឺជាកម្មវិធីលើគេហទំព័រ *(web-based application)* ដែលត្រូវបានរចនាឡើងដើម្បីបម្លែងកាយវិការនៃភាសាសញ្ញាខ្មែរ ទៅជាអត្ថបទដែលអាចអានបាន ដោយប្រើប្រាស់បច្ចេកវិទ្យា *computer vision* និង *machine learning*។ គម្រោងនេះមានគោលបំណងកាត់បន្ថយរបាំងទំនាក់ទំនងរវាងអ្នកប្រើ ប្រាស់ភាសាសញ្ញា និងអ្នកដែលមិនប្រើភាសាសញ្ញា តាមរយៈការបកប្រែភ្លាមៗ (*real-time gesture recognition)* ដោយមាន *web interface* ដែលងាយស្រួលប្រើ និងអាចចូលដំណើរការបាន។ ប្រព័ន្ធនេះប្រើការម៉ាកេ *(webcam)* ដើម្បីចាប់យកវីដេអូពី កាម៉េរ៉ាផ្ទាល់ៗ *(live video input)* បន្ទាប់មកចាប់យក និងដំណើរការទៅលើកាយវិការទាំងនោះ *(hand gestures)* ដើម្បីបង្ហាញ លទ្ធផលជាអត្ថបទ ដែលជួយធ្វើឱ្យការទំនាក់ ទំនងកាន់តែងាយស្រួល និងមានប្រសិទ្ធភាពបានល្អជាងមុន។

ដើម្បីសម្រេចបានគូរមុខងារទាំងនេះ ប្រព័ន្ធរបស់យើងបានប្រើ *MediaPipe* សម្រាប់ការចាប់យកចំណុចនៅលើដៃ *(hand landmark detection)* ដែលមានភាពត្រឹមត្រូវខ្ពស់ និង *deep learning model* ដែលបានកែលម្អដោយប្រើ *PyTorch* ដើម្បីសនិដ្ឋានពីកាយវិកា *(classify gesture)* ភាសាសញ្ញាខ្មែរ ចំនួន ៣៨ ពាក្យ។ ចំណុចនៅលើដៃ *(Hand landmarks)* ដែលទាញយកមកបាន *(extract)* ត្រូវបានវិភាគភ្លាមៗ *(real time)* ធ្វើឱ្យ *model* អាចធ្វើការព្យាករណ៍ *(prediction* បានយ៉ាងលឿន និងមានភាពយឺតយ៉ាវ *(latency)* តិចជាងមួយវិនាទី។ ក្នុងដំណាក់កាលសនិដ្ឋាន *(evaluation),* ម៉ូឌែល *(model)* បានសម្រេចនូវ *validation accuracy* រហូតទៅដល់ ៨៧.៩៩% ដែលបង្ហាញពីប្រសិទ្ធភាព *(performance)* ល្អលើកាយវិការ *(gesture classes)* ទាំងអស់។ ការសម្រេចបានចុងក្រោយរបស់យើង គឺបានបង្កើតជាគេហទំព័រ ដែលជាផលិតផលគំរូ *(web-based prototype)* ដែលដំណើរការបានពេញលេញ និងបានរួមបញ្ចូលបច្ចេកវិទ្យា *computer vision, machine learning* និង *web technologies* យ៉ាងជោគជ័យ។ របាយការណ៍នេះបង្ហាញលម្អិតអំពីដំណើរការនៃការអភិវឌ្ឍ *(development process)* ទាំងមូល រួមមាន *system architecture, training methodology, technical implementation* និង *evaluation results* ដោយបញ្ជាក់ពីរបៀប ដែលបញ្ញាសិប្បនិម្មិត *(artificial intelligence)* អាចត្រូវបានអនុវត្តជាក់ស្តែង ដើម្បីបន្ថែមនូវភាពងាយស្រួល *accessibility* និងគាំទ្រសហគមន៍ដែលមិនទទួល បានសេវាកម្មគ្រប់គ្រាន់។

The **Khmer Sign Language Translation System** is a web-based application designed to convert Khmer sign language gestures into readable text using computer vision and machine learning techniques. The project aims to bridge the communication gap between sign language users and non–sign language users by enabling real-time gesture recognition through an intuitive and accessible web interface. The system leverages a webcam to capture live video input, from which hand gestures are detected and processed to produce corresponding textual output, making communication more inclusive and efficient.

To achieve this functionality, the system utilizes MediaPipe for accurate hand landmark detection and a refined deep learning model built with PyTorch to classify more than 38 distinct Khmer sign language gestures. The extracted hand landmarks are analyzed in real time, allowing the model to make fast and reliable predictions with sub-second latency. During evaluation, the model achieved a validation accuracy of 87.99%, demonstrating strong performance across all gesture classes. The final implementation resulted in a fully functional web-based prototype that successfully integrates computer vision, machine learning, and modern web technologies. This report presents the complete development process, including system architecture, training methodology, technical implementation, and evaluation results, highlighting how artificial intelligence can be effectively applied to improve accessibility and support underserved communities.

Keywords: *Sign Language Translation, Computer Vision, Machine Learning, MediaPipe, PyTorch, Real-time Recognition, Accessibility Technology, Web Application*

Table of Contents

List of Figures

List of Tables

List of Abbreviations & Acronyms

- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **CNN**: Convolutional Neural Network
- **CORS**: Cross-Origin Resource Sharing
- **CPU**: Central Processing Unit
- **CUDA**: Compute Unified Device Architecture
- **FPS**: Frames Per Second
- **GPU**: Graphics Processing Unit
- **HTTP**: Hypertext Transfer Protocol
- **JSON**: JavaScript Object Notation
- **ML**: Machine Learning
- **NLP**: Natural Language Processing
- **REST**: Representational State Transfer
- **TTS**: Text-to-Speech
- **UI**: User Interface
- **UX**: User Experience
- **WCAG**: Web Content Accessibility Guidelines

Chapter 1

Introduction

1.1. Project Overview

The Khmer Sign Language Translation System is a web-based program created as a capstone project for third-year students in the Computer Science and Data Science departments. By converting Khmer sign language gestures into readable text output via a straightforward web interface, this system aims to facilitate communication.

The system is an example of how artificial intelligence and computer vision technologies can be used practically to solve accessibility issues in the real world. The project shows how technical expertise can be used to develop significant solutions for marginalized communities by fusing machine learning with contemporary web technologies.

1.2. Problem Statement

In everyday interactions, communication barriers between deaf and hard-of-hearing individuals who rely on sign language and those who do not understand it present significant challenges. Learning sign language requires considerable time and effort, while professional interpretation services are often limited in availability and costly. These challenges are particularly evident in educational and professional environments, where timely and effective communication is essential. Furthermore, accessible translation tools for regional sign languages such as Khmer sign language remain scarce. This project addresses these issues by proposing an automated, web-based system that supports real-time recognition of a limited set of static Khmer sign language gestures, thereby providing a more accessible and affordable communication aid without requiring specialized hardware.

1.3. Technical Approach

This project uses a Khmer sign language dataset created by the team to refine a pre-trained model rather than creating the model from scratch. This method shortens the development time while improving the system's ability to identify local gestures. To enhance performance and usability, the team concentrates on data processing, model optimization, and system integration.

The system recognizes hand gestures in real time using computer vision and machine learning techniques. When a user signs in front of a camera, MediaPipe is used by the system to detect hand movements, and PyTorch is used to build a trained deep learning model to process the gesture. To facilitate communication with non-sign language users, the anticipated outcome is then shown as text.

1.4. Project Status

The system might not always generate flawless translations because this project is still in active development. Future updates will include additional gestures, and accuracy and coverage are constantly being enhanced. The current version shows a working prototype that illustrates how AI can enhance learning and accessibility.

1.5. Objectives

1.5.1. Primary Objectives

This project's main goal is to create a web-based Khmer Sign Language Translation System that can identify hand gestures and instantly translate them into readable text and spoken output. By integrating computer vision and machine learning technologies into an accessible web application, the system seeks to function as a communication tool as well as a learning tool.

1.5.2. Secondary Objectives

This project also focuses on improving the understanding of artificial intelligence implementation in real-world scenarios. By working with datasets, pre-trained models, and system integration, the team gains hands-on experience in building practical machine learning applications that go beyond theory.

1.5.3. Specific Objectives

1. **To design a real-time gesture recognition system** that can detect and interpret Khmer sign language movements using a webcam.
2. **To translate recognized gestures** into meaningful and readable text on the screen.
3. **To fine-tune an existing machine learning model** using a Khmer sign language dataset to improve accuracy and local relevance.
4. **To build a responsive and user-friendly web interface** that can be accessed easily through a browser.
5. **To optimize the system** for smooth performance and reduced delay during real-time translation.
6. **To test the system** under different conditions such as lighting and hand position.
7. **To collect feedback from users** to identify weaknesses and areas for improvement.
8. **To expand the number of supported signs** as data becomes available.

1.6. Scope of the Project

This project focuses on the real-time recognition of static Khmer sign language gestures and the translation of recognized gestures into readable text. The system is implemented as a web-based application that can be accessed through modern web browsers without requiring any software installation. It supports a predefined set of 38 commonly used static Khmer sign language gestures and is designed for single-user detection, allowing one individual to perform gestures at a time. To ensure reliable performance and manageable system complexity, the project intentionally limits its scope to static hand gestures, with motion-based gestures identified as potential future work.

Several features are considered outside the scope of this project. The system does not support dynamic gestures involving continuous motion, text-to-speech conversion for audio output, or simultaneous detection of multiple users. Offline functionality is not provided, as the system relies on real-time processing through a web interface. In addition, native mobile applications are not developed, and reverse translation from text to sign language is not addressed. These limitations are defined to maintain a clear project focus and ensure feasibility within the given academic timeframe.

1.7. Significance of the Study

This project contributes to:

1. **Accessibility Technology:** Advances in assistive technology for the deaf and hard-of-hearing community

2. **Khmer Language Support:** Addresses the lack of resources for Khmer sign language
3. **Educational Value:** Demonstrates practical application of AI and ML in real-world scenarios
4. **Technical Innovation:** Shows integration of computer vision, ML, and web technologies
5. **Research Contribution:** Provides a dataset and model for Khmer sign language recognition

1.8. Report Organization

This report is organized into eleven chapters:

- **Chapter 1:** Introduction and project overview
- **Chapter 2:** Literature review and background research
- **Chapter 3:** System overview and architecture
- **Chapter 4:** Frontend design and implementation
- **Chapter 5:** Backend design and implementation
- **Chapter 6:** Data and model development
- **Chapter 7:** Model evaluation and results
- **Chapter 8:** Testing and validation
- **Chapter 9:** Discussion of results and limitations
- **Chapter 10:** Conclusion and future work
- **Chapter 11:** Project management and collaboration

Chapter 2

Literature Review & Background

2.1. Related Work

MediaPipe Hands provides real-time hand landmark detection and has been widely used in gesture recognition systems due to its low latency and cross-platform support [1]. OpenPose is another widely cited framework for multi-person pose estimation, although it is computationally heavier for real-time web applications [14].

2.1.1. Specific Objectives

Several approaches have been used for sign language recognition:

1. **Vision-Based Approaches:** Using cameras to capture hand movements and gestures[17][18][19][20][21]
2. **Sensor-Based Approaches:** Using gloves or sensors to track hand positions[16]
3. **Hybrid Approaches:** Combining vision and sensor data[6]

2.1.2. Deep Learning in Sign Language Recognition

Recent advances in deep learning have significantly improved sign language recognition accuracy:

- **Convolutional Neural Networks (CNNs)** for image-based recognition
- **Recurrent Neural Networks (RNNs)** for temporal gesture sequences
- **Transformer Models** for sequence-to-sequence translation
- **Transfer Learning** for adapting pre-trained models to sign language

2.2. Existing Systems

2.2.1. Commercial Solutions

Several commercial sign language translation solutions have been developed using artificial intelligence and computer vision technologies. **SignAll** is an AI-powered sign language translation system designed to convert sign language into text or speech, primarily for institutional and educational use cases [22]. **MotionSavvy** provides real-time sign language recognition using depth-sensing cameras and wearable hardware, with a strong focus on American Sign Language (ASL) [23]. **Hand Talk** is a mobile application that translates spoken or written language into sign language using an animated avatar and supports several major sign languages, mainly for accessibility and customer service purposes [24].

Despite their technological advancements, these commercial solutions exhibit notable limitations. Most systems primarily target ASL or other widely used sign languages, offering limited or no support for regional sign languages such as Khmer sign language. Additionally, some solutions depend on specialized or proprietary hardware, increasing system complexity and limiting scalability.

Furthermore, the high licensing and deployment costs associated with commercial platforms reduce accessibility, particularly in developing regions. These limitations emphasize the need for a cost-effective, web-based sign language translation system specifically designed for Khmer sign language.

2.2.2. Research Systems

- **MediaPipe Hands:** Google's hand tracking solution [1]
- **OpenPose:** Multi-person pose estimation [14]
- **Sign Language Recognition Datasets:** Various academic datasets [25][26]

2.3. Technologies & Frameworks Review

(See Appendix C for the Complete Comparison of Technology Stack in table C.1)

2.3.1. Computer Vision Libraries

2.3.2. Machine Learning Frameworks

MediaPipe is an open-source framework developed by Google that enables real-time hand tracking and landmark detection. It provides 21 three-dimensional hand landmarks per detected hand, which represent key joint positions and fingertips. MediaPipe is optimized for cross-platform deployment and offers low-latency performance, making it suitable for real-time applications such as gesture and sign language recognition. In this project, MediaPipe is used to extract hand landmark coordinates from live webcam input, which serve as the primary features for gesture classification.

OpenCV is a widely used open-source computer vision library that provides essential tools for image processing and video analysis. It supports real-time video capture, frame preprocessing, and the implementation of classical computer vision algorithms. In this system, OpenCV is utilized to handle webcam input, perform frame resizing and color space conversion, and facilitate efficient communication between the video stream and the gesture recognition pipeline.

PyTorch is an open-source deep learning framework widely used in academic research due to its dynamic computation graph, which allows models to be defined and modified during runtime. This flexibility makes PyTorch particularly suitable for rapid experimentation, debugging, and iterative model development. In addition, PyTorch provides built-in support for GPU acceleration, enabling faster training and inference for neural network models. In this project, PyTorch is selected as the primary framework for training the hand gesture classification model, as it facilitates efficient experimentation with landmark-based features and supports quick model refinement during development.

TensorFlow is another widely adopted deep learning framework that is commonly used in large-scale and production-ready systems. It offers a comprehensive ecosystem of tools for model deployment, including TensorFlow.js, which enables machine learning models to run directly in web browsers. While TensorFlow is well-suited for production environments, this project prioritizes research flexibility and rapid prototyping over large-scale deployment. As a result, PyTorch is preferred for model development, while TensorFlow is not used in the implementation of this system.

Selection Rationale: PyTorch was chosen for its flexibility in model development and fine-tuning, which is essential for this research project.

2.3.3. Web Technologies

React is a widely used JavaScript library for building user interfaces based on a component-based architecture. This design approach allows the user interface to be divided into reusable and maintainable

components, improving development efficiency and code organization. React also offers good rendering performance through its virtual DOM mechanism and benefits from a large ecosystem of libraries and tools. Due to its strong community support and suitability for building responsive web interfaces, React is used in this project to develop the frontend of the web-based Khmer sign language recognition system.

Flask is a lightweight Python web framework that is well suited for developing backend services and RESTful APIs. Its minimal design and flexibility make it easy to integrate with machine learning models developed in Python. In this project, Flask is used to implement the backend API that handles communication between the frontend interface and the gesture recognition model. Flask enables efficient handling of prediction requests and supports real-time interaction between the web client and the machine learning components of the system.

2.4. Gaps in Current Approaches

Most existing sign language recognition systems focus on comprehensive vocabularies and dynamic gesture sequences, which require large datasets and complex temporal modeling. In contrast, Khmer sign language remains underrepresented in both academic research and practical systems, particularly for lightweight, web-based solutions. Additionally, many existing approaches rely on native applications or specialized hardware, limiting accessibility.

This project deliberately limits its scope to static Khmer sign language gestures commonly used in daily communication. By focusing on a subset of 38 signs and using hand landmark-based recognition, the system prioritizes real-time performance, accessibility, and feasibility within the constraints of a student capstone project. This scoped approach allows for effective evaluation while establishing a foundation for future expansion.

Our Contribution: This project addresses these gaps by providing an open-source, web-based system for real-time recognition of **38 predefined static Khmer sign language gestures**, using webcam-based hand landmark detection without requiring specialized hardware.

Chapter 3

Project Management & Collaboration

3.1. Development Methodology

The project follows an iterative development approach combining agile principles with academic research methodology:

3.1.1. Agile Development

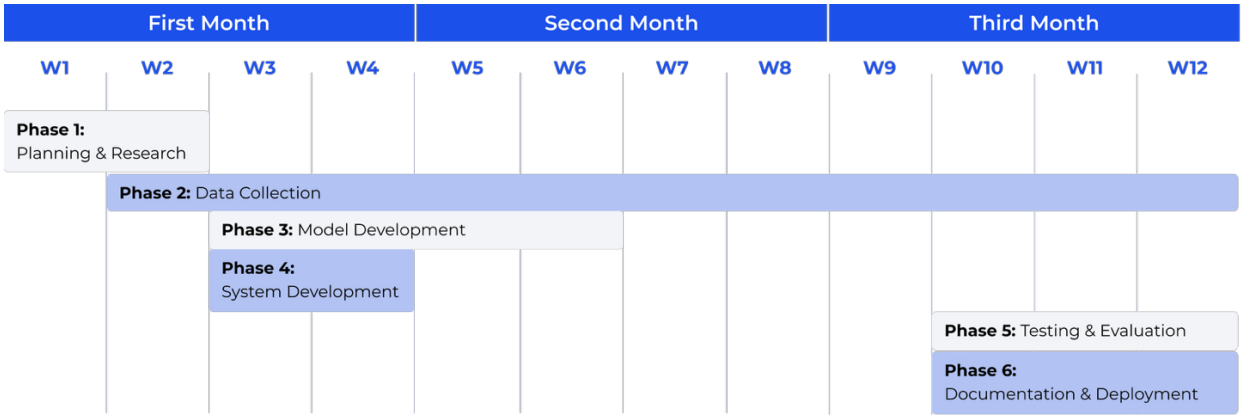
- **Short Development Cycles:** 2-week sprints with regular milestones
- **Continuous Integration:** Regular code integration and testing
- **User-Centered Design:** Regular user feedback and iteration
- **Adaptive Planning:** Flexible planning based on progress and feedback

3.1.2. Research Methodology

- **Literature Review:** Comprehensive review of existing systems
- **Experimental Design:** Systematic approach to model development
- **Data Collection:** Structured data collection protocols
- **Evaluation:** Rigorous evaluation and validation

3.2. Timeline & Milestones

3.2.1. Project Phases



(Figure 3.1: Project Timeline)

Phase 1: Planning & Research (Weeks 1-2)

- Project proposal and planning
- Literature review
- Technology selection
- Requirements analysis

Phase 2: Data Collection (Weeks 2-12)

- Data collection protocol design
- Gesture recording and collection
- Data preprocessing and labeling
- Dataset validation

Phase 3: Model Development (Weeks 3-6)

- Model architecture design
- Training pipeline development
- Model training and optimization
- Initial evaluation

Phase 4: System Development (Weeks 3-4)

- Frontend development
- Backend development
- API integration
- System integration

Phase 5: Testing & Evaluation (Weeks 10-12)

- Unit testing
- Integration testing
- User acceptance testing
- Performance evaluation

Phase 6: Documentation & Deployment (Weeks 10-12)

- Documentation writing
- Report preparation
- System deployment
- Final presentation

3.2.2. Key Milestones

- ✓ Project proposal approval
- ✓ Dataset collection completion
- ✓ Model training completion (87.99% accuracy)
- ✓ Frontend development completion
- ✓ Backend development completion
- ✓ System integration completion
- ✓ User testing completion
- ✓ Final report submission

3.3. Tools for Version Control & Collaboration

3.3.1. Version Control

- **Git:** Distributed version control system
- **GitHub:** Code repository and collaboration platform
- **Branching Strategy:** Feature branches with main/master branch
- **Commit Standards:** Conventional commit messages

3.3.2. Collaboration Tools

- **Communication:** Discord, Email, Meetings
- **Project Management:** GitHub Issues, Project Boards
- **Documentation:** Markdown, README files, Docx files
- **Code Review:** GitHub Pull Requests

3.3.3. Development Tools

- **IDE:** VS Code, PyCharm
- **Design:** Figma (for UI/UX design)
- **Testing:** Manual testing, User feedback
- **Deployment:** Vercel (frontend), Local server (backend)

3.4. Team Contributions

3.4.1. Team Structure

The project team consists of six members with specialized roles:

Team Member	Role	Primary Responsibilities	Key Contributions
Vann Vat	Team Leader & Data Engineer	Project coordination, Data management	Project oversight, Data collection and preprocessing, Dataset organization, Quality control
Chim Panhaprasith	Frontend Developer	React development, UI implementation	React application development, User interface design, Frontend-backend integration, Responsive design
Chhi Hangcheav	Backend Developer	Flask API, Server development	Flask API development, Model integration, Server configuration, API endpoint design
Toek Hengsreng	Dataset & Research Analyst	Data collection, Research	Data collection and labeling, Literature review, Dataset analysis, Quality assurance
Mony Meakputsoktheara	Machine Learning Engineer	Model development, Training	Model architecture design, Training pipeline, Model optimization, Performance tuning

Phal Sovandy	UI/UX Designer & Content Lead	Design, Documentation	User interface design, User experience optimization, Documentation creation, Visual design

(Table 3.1: Team Contributions)

3.4.2. Individual Contributions

Vann Vat - Team Leader & Machine Learning Engineer

- Project coordination and management
- Overall project oversight
- Model architecture design
- Training pipeline development
- Model optimization and evaluation
- Performance tuning

Chim Panhaprasith - Frontend Developer

- React application development
- User interface design and implementation
- Frontend-backend integration
- Responsive design implementation

Chhi Hangcheav - Backend Developer

- Flask API development
- Model integration and optimization
- Server configuration and deployment
- API endpoint design

Toek Hengsreng - Dataset & Research Analyst

- Data collection and labeling
- Research and literature review
- Dataset analysis and validation
- Quality assurance

Mony Meakputsoktheara - Data Engineer

- Data collection and preprocessing
- Dataset organization and quality control
- Feature extraction and data formatting
- Preparation of structured datasets

Phal Sovandy - UI/UX Designer & Content Lead

- User interface design
- User experience optimization
- Documentation and content creation
- Visual design and branding

3.4.3. Collaborative Efforts

- **Code Reviews:** All team members participated in code reviews

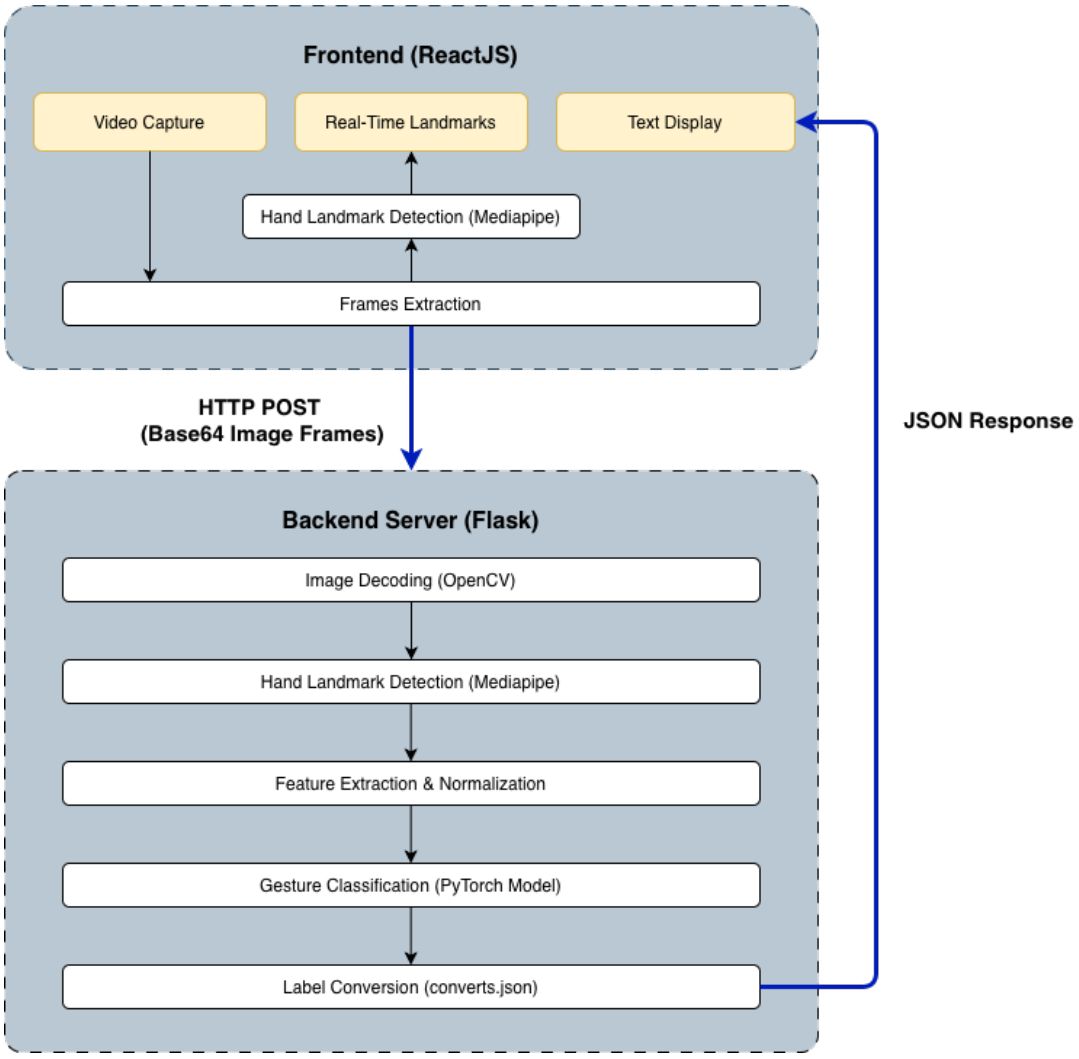
- **Testing:** Collaborative testing and bug fixing
- **Documentation:** Shared documentation responsibilities
- **Presentations:** Team presentations and demos

Chapter 4

System Overview

4.1. Overall System Architecture

The Khmer Sign Language Translation System processes user gestures through a series of stages that convert visual hand movements into readable text and spoken output. The system combines computer vision techniques with a trained machine learning model to provide real-time translation through a web-based interface.

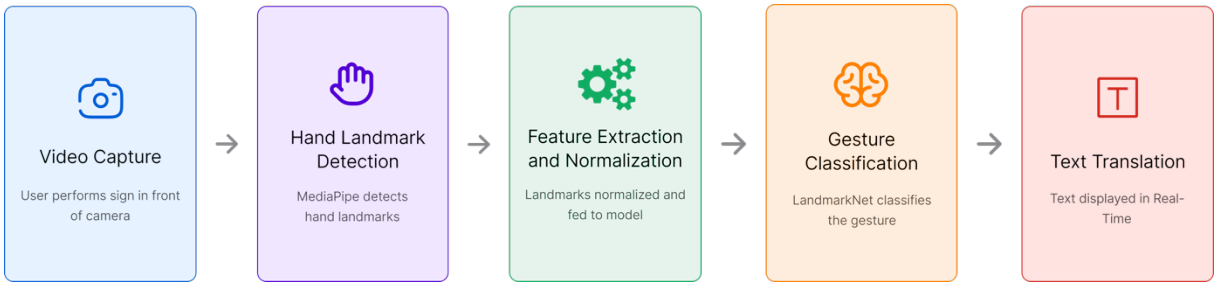


(Figure 4.1: System Interaction Flow Overview Diagram)

(See Appendix B for the System Interaction Sequence Diagram in diagram B.4.)

4.2. How the System Works

4.2.1. Processing Pipeline



(Figure 4.2: End-to-End Process Overview)

Stage 1: Video Capture

In front of the camera on their device, the user first makes gestures in Khmer sign language. Video frames are continuously captured by the web application and sent to the processing system. The user must make clear gestures and keep their hands in the camera's field of view in order to guarantee accurate tracking.

The user's webcam is accessed by the frontend application via the browser's MediaStream API, which records frames at regular intervals (usually 30 frames per second). Before being transmitted to the backend server, each frame is transformed into a base64-encoded image format.

Stage 2: Hand Landmark Detection

The system then tracks and detects the user's hands using MediaPipe. Important hand landmarks like finger joints and palm positions are recognised by MediaPipe. Instead of using raw images, the system can comprehend the shape and movement of the hand thanks to these landmarks, which represent the gesture in numerical form. This step decreases background detail noise and speeds up processing.

MediaPipe Hands provides 21 hand landmarks per hand:

- 4 landmarks for each finger (thumb, index, middle, ring, pinky)
- 1 landmark for the wrist

Each landmark is represented by normalized coordinates `(x, y)` relative to the image dimensions, making the detection scale-invariant.

Example of hand landmark detection using MediaPipe:

- The system extracts 38 features per hand (21 landmarks × 2 coordinates)
- For two-hand gestures, the system concatenates features from both hands (84 total features)
- Landmark coordinates are normalized to improve model generalization

Stage 3: Feature Extraction and Normalization

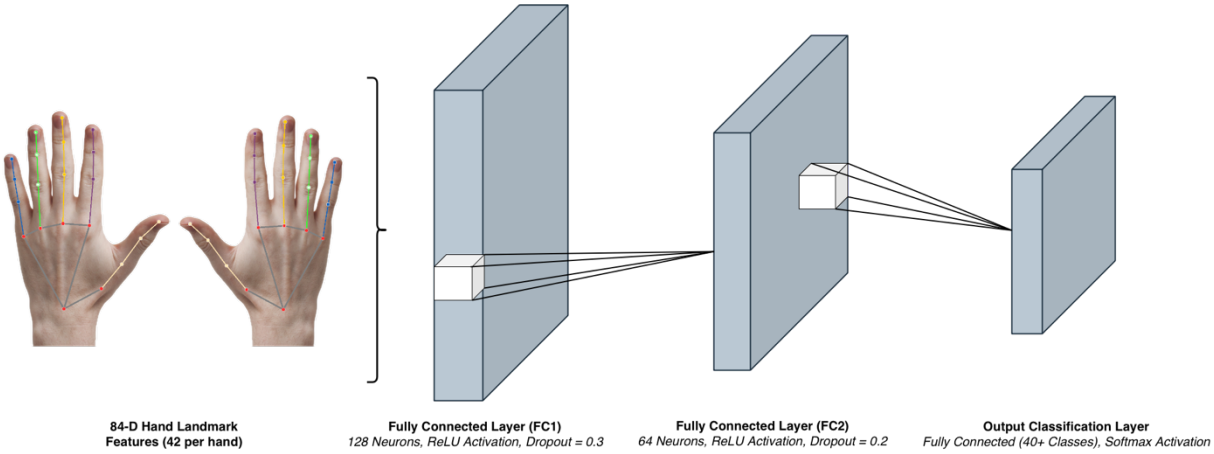
Once the hand landmarks are detected, the extracted features are normalized to ensure consistent input to the machine learning model. The normalization process includes:

- **Mean Centering**: Subtracting the mean of landmark coordinates
- **Standard Scaling**: Dividing by the standard deviation to normalize the scale
- **Feature Concatenation**: Combining `x` and `y` coordinates into a single feature vector

This normalization process makes the model robust to variations in hand size, position, and camera distance.

Stage 4: Gesture Classification

A PyTorch-built deep learning model receives the normalised features. A dataset of Khmer sign language gestures has been used to refine the model, enabling it to recognise patterns unique to regional signs. The model determines which gesture is most likely to be made by analysing the landmarks.



(Figure 4.3: Model Architecture - LandmarkNet)

Model Architecture:

- **Input Layer**: 84 features (42 per hand × 2 hands)
- **Hidden Layer 1**: 128 neurons with ReLU activation and 0.3 dropout
- **Hidden Layer 2**: 64 neurons with ReLU activation and 0.2 dropout
- **Output Layer**: Number of classes (38 gesture classes)

The model outputs a probability distribution over all possible gesture classes, and the class with the highest probability is selected as the prediction.

Stage 5: Text Translation

After prediction, the system converts the recognized sign into text and displays it in real time on the interface. This allows the user to immediately see translation results as they perform gestures.

The system uses a conversion mapping file (`converts.json`) to translate model predictions (form names like “form_a”) into human-readable Khmer text. This mapping ensures that the output is meaningful and culturally appropriate.

4.2.2. Data Flow Diagram

```
User Camera → Frontend (React) → Base64 Image → Backend (Flask)
                                                      ↓
                                            MediaPipe Hand Detection
                                                      ↓
                                          Feature Extraction (84 features)
                                                      ↓
                                            PyTorch Model Inference
                                                      ↓
                                            Gesture Classification
                                                      ↓
                                          Label Conversion (converts.json)
```

(Figure 4.4: Data flow Diagram)

4.2.3. Real-Time Performance

The system is designed and optimized for real-time gesture recognition. Average latency ranges between 10 and 50 milliseconds per frame on CPU-based processing, and between 5 and 20 milliseconds per frame when utilizing GPU acceleration. The application is capable of processing up to 30 frames per second, ensuring smooth visualization of hand gestures. Throughput has been optimized to handle multiple concurrent requests without performance degradation. Furthermore, resource usage has been minimized, allowing the system to operate efficiently on standard hardware without requiring specialized GPUs. These performance optimizations ensure that users experience real-time feedback while maintaining accessibility and usability across typical desktop and laptop environments.

4.3. Functional Requirements

1. **Gesture Recognition:** Recognize 38 Khmer sign language gestures
2. **Real-Time Processing:** Process gestures in real-time (<1s latency)
3. **Text Display:** Display recognized gestures as Khmer text
4. **User Interface:** Provide intuitive web-based interface
5. **Settings Configuration:** Allow users to adjust sensitivity settings
6. **Error Handling:** Gracefully handle errors and provide feedback

4.4. Non-Functional Requirements

1. **Performance:** Sub-Second latency for real-time recognition
2. **Usability:** Intuitive interface requiring minimal training
3. **Accessibility:** WCAG 2.1 compliance for accessibility
4. **Compatibility:** Support for modern web browsers
5. **Scalability:** Handle multiple concurrent users
6. **Reliability:** 99%+ uptime for production deployment
7. **Security:** Secure handling of user data and video streams

4.5. System Requirements (Hardware & Software)

4.5.1. Hardware Requirements

The following hardware requirements are recommended for optimal performance:

- **Computer or Laptop**: A computer or laptop with a working webcam for capturing sign language gestures.
- **Memory**: Minimum 4 GB of RAM to allow smooth browser operation during real-time processing.
- **Processor**: A modern processor capable of handling video input without lag.
- **Lighting**: Stable lighting conditions in the user's environment to improve hand detection accuracy.

4.5.2. Software Requirements

To access and use the system, users need:

- **Web Browser**: A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox.
- **Internet Connection**: An active internet connection since the application processes data through a web interface.
- **JavaScript**: JavaScript enabled in the browser.
- **Operating System**: A supported operating system such as Windows, macOS, or Linux.

4.5.3. User Requirements

Users should:

- Allow access to the device camera in the browser.
- Perform gestures clearly and within the camera frame.
- Maintain an appropriate distance from the camera for accurate detection.
- Operate the system in an environment with sufficient lighting.

4.5.4. Development Requirements

- **Python 3.11+:** For backend development
- **Node.js 18.0+:** For frontend development
- **PyTorch 2.1.0+:** For model inference
- **MediaPipe 0.10.8+:** For hand tracking
- **React 19.2.0+:** For frontend framework
- **Flask 3.0.0+:** For backend server

4.5.5. Development Requirements

- **Frontend**: React-based single-page application with real-time video processing
- **Backend**: Flask REST API server for model inference
- **Model Storage**: Trained PyTorch model files (`.pth` format)
- **Data Storage**: JSON configuration files for gesture mappings

4.6. Methodology

This project follows a systematic approach to developing a sign language translation system, combining research, development, and evaluation phases.

Phase 1: Research and Planning

The first phase focused on research and project planning. A comprehensive literature review was conducted to examine existing sign language recognition systems, computer vision techniques for hand tracking, and machine learning approaches for gesture recognition. In addition, relevant accessibility technology standards were reviewed to ensure inclusivity considerations. Based on the findings, various computer vision libraries such as MediaPipe and OpenCV were evaluated, along with deep learning frameworks including PyTorch and TensorFlow. Web technologies suitable for real-time processing were also assessed. The final technology stack was selected based on performance, compatibility, and project requirements. During this phase, user needs and use cases were identified, system requirements and constraints were defined, success criteria and evaluation metrics were established, and a project timeline with key milestones was planned.

Phase 2: Data Collection and Preparation

The second phase involved data collection and preparation. A data collection protocol was designed to ensure consistency and quality across gesture samples. Team members and volunteers participated in the data collection process, and quality standards were established for capturing static Khmer sign language gestures. A structured data labeling and organization system was created to support efficient training and evaluation. MediaPipe was implemented to extract hand landmark coordinates from the collected data. Preprocessing steps such as normalization and data augmentation were applied, and validation procedures were introduced to maintain data quality. The dataset was then organized into training, validation, and testing subsets.

Phase 3: Model Development

In the model development phase, a landmark-based neural network architecture referred to as LandmarkNet was designed. Appropriate layer sizes and activation functions were selected, and dropout was incorporated to reduce overfitting. The model was optimized for real-time inference to meet performance requirements. Training was implemented using PyTorch, with careful configuration of hyperparameters such as learning rate and batch size. Training progress and validation metrics were continuously monitored, and early stopping techniques were applied to prevent overfitting. Following training, the model was further optimized through quantization to reduce size and improve inference speed. Performance was tested across different hardware configurations, and evaluation metrics were validated.

Phase 4: System Integration

The system integration phase focused on combining frontend and backend components into a unified application. A React-based user interface was developed to support real-time video capture, hand tracking, and text-based output display. MediaPipe was integrated into the frontend to enable real-time hand landmark detection. On the backend, a Flask-based RESTful API was developed and integrated with the trained PyTorch model to perform gesture inference. Cross-Origin Resource Sharing (CORS) was configured to allow secure communication between the frontend and backend. The complete system was tested end-to-end to ensure real-time data flow, correct predictions, and optimized overall performance.

Phase 5: Testing and Evaluation

Testing and evaluation were conducted to assess system correctness, usability, and performance. Unit testing was performed on individual components, including the machine learning model, API endpoints, and user interface. Data preprocessing pipelines and model inference accuracy were verified. Integration testing validated communication between the frontend and backend and ensured reliable real-time processing. User testing sessions were conducted to evaluate usability and collect feedback, which was used to identify areas for improvement. Performance evaluation measured inference latency, accuracy on the test dataset, system behavior under different conditions, and resource usage.

Phase 6: Documentation and Deployment

The final phase focused on documentation and deployment. User documentation, technical documentation, and API documentation were created to support system usage and maintenance. A comprehensive project report was prepared to document the development process and results. For deployment, a production environment was configured, the frontend was deployed to a web hosting platform, and the backend server was set up to handle inference requests. System performance was monitored to ensure stability and responsiveness after deployment.

4.6.1. Evaluation Methodology

Quantitative Evaluation

Accuracy Metrics:

The system's performance was assessed using standard quantitative metrics. Classification accuracy on the validation dataset reached **87.99%**, indicating a high degree of predictive reliability. Further analysis was conducted on a per-class basis, including **precision, recall, and F1-scores**, providing insights into the model's ability to correctly identify individual gesture classes. A **confusion matrix** was constructed to visualize misclassification patterns and identify classes requiring further improvement.

Performance Metrics:

Performance efficiency was also measured. The inference latency ranged from 10–50 milliseconds per frame on CPU-based computation, supporting a frame processing rate of 30 FPS, which meets real-time requirements. Resource utilization, including CPU and memory consumption, was monitored, and the system demonstrated effective handling of concurrent requests, reflecting its throughput capacity.

Qualitative Evaluation

User Experience:

The user experience and system robustness were evaluated qualitatively. **Usability testing sessions** and **user feedback surveys** were conducted to assess interface intuitiveness, ease of use, and overall satisfaction. The design was further analyzed in terms of **interface aesthetics** and **accessibility features** to ensure inclusivity.

System Robustness:

System robustness was examined under diverse conditions. The model was tested with **different lighting conditions, varying hand positions and sizes, and diverse background environments**. Additionally, it was evaluated across **different user signing styles**, ensuring reliable performance under real-world variability.

4.6.2. Ethical Considerations

Privacy:

The system prioritizes user privacy by ensuring that **camera data is processed locally whenever possible**, minimizing the need to transmit sensitive video data. **No user video data is stored**, and all operations comply with a clearly stated **privacy policy**, with explicit **user consent** obtained prior to usage.

Accesibility:

The design follows inclusive design principles and established accessibility guidelines, ensuring usability for individuals with disabilities. Interface elements and interaction workflows were evaluated to guarantee that the system is accessible to a broad range of users, including those with physical or sensory impairments.

Bias and Fairness:

To mitigate algorithmic bias, the dataset was **collected from a diverse user population**, representing a variety of demographic groups. System performance was **tested across different user demographics** to ensure fairness. Furthermore, mechanisms for **continuous monitoring of bias** are in place to maintain equitable system behavior over time.

Chapter 5

Frontend Design & Implementation

5.1. Frontend Overview

Modern web technologies are used in the frontend of the Khmer Sign Language Translation System to create a user interface that is accessible, responsive, and easy to use. The frontend, which manages video capture, real-time display, and user feedback, acts as the main user interface.

5.1.1. Frontend Responsibilities

The gesture recognition system is composed of several key functional components designed to ensure both usability and performance:

User Interface – The system provides an **intuitive and accessible interface** that allows users to interact seamlessly with the gesture recognition features. Interface design emphasizes clarity, ease of navigation, and adherence to accessibility standards.

Video Capture – The system manages **webcam access and video stream processing**, ensuring smooth acquisition of hand movements while maintaining privacy and low latency.

Real-Time Display – **Hand landmarks, gesture predictions, and real-time feedback** are visualized on the interface to provide users with immediate insights into system operation and performance.

User Interaction – The system handles **user inputs, configuration settings, and control mechanisms**, allowing users to customize their experience and interact efficiently with the recognition engine.

Visual Feedback – Recognized gestures, associated **confidence scores**, and **system status indicators** are displayed clearly, supporting transparency and helping users understand the system's decisions.

5.2. User Interface & UX Design Principles

5.2.1. Design Principles

The system interface was developed following a set of core design principles to enhance usability, accessibility, and user engagement.

Accessibility-First Approach
The interface was designed in alignment with **WCAG 2.1 guidelines**, ensuring that users with diverse abilities can effectively interact with the system.

Responsive Design
The application supports **responsive layouts**, allowing seamless operation across **desktop, tablet, and mobile platforms** without loss of functionality or usability.

Intuitive Navigation
A **clear navigation structure** and well-defined user flows were implemented to reduce cognitive load and improve user efficiency.

Visual Feedback
The system provides **immediate visual feedback** in response to user actions, improving transparency and user confidence during interaction.

Modern Aesthetics
A **clean, modern visual design** complemented by subtle animations was adopted to enhance user experience while maintaining clarity and professionalism.

5.2.2. Key UI Features

Demo Page Interface

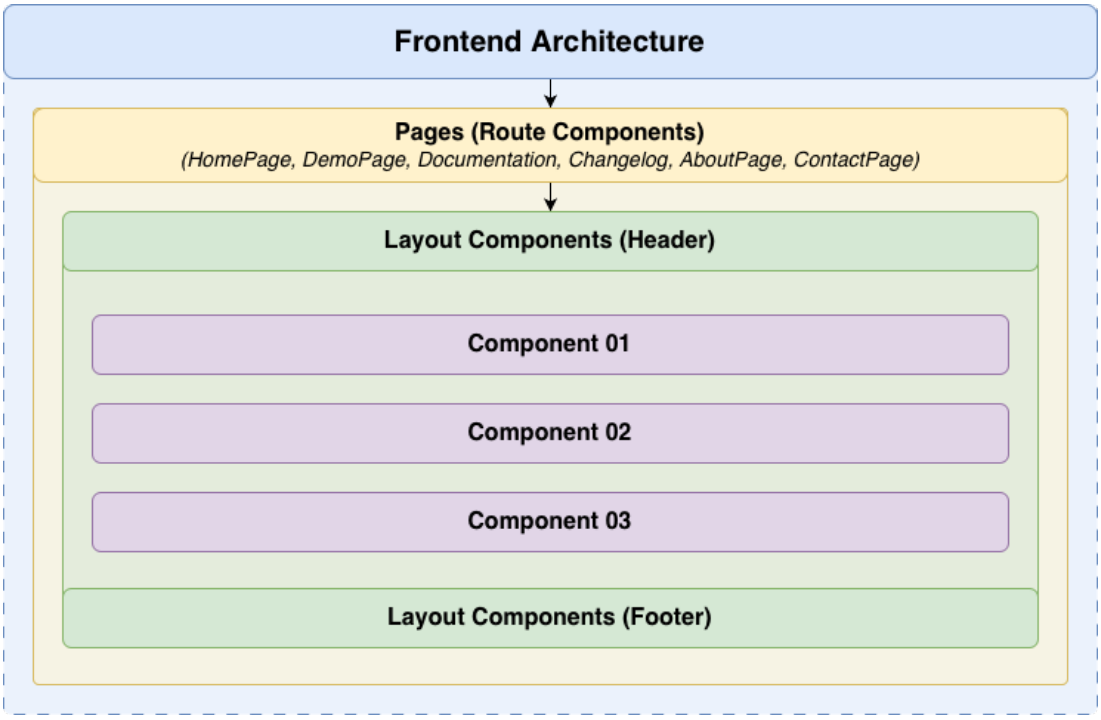
The demo page was designed to support real-time interaction and system transparency. It features a **live video feed** integrated with **hand landmark visualization**, allowing users to observe gesture tracking directly. Recognized gestures are displayed in a **dedicated text output area**, providing immediate interpretation feedback. User controls, including **camera activation and system settings**, are accessible through clearly labeled control buttons. A **settings modal** enables customization of system parameters, enhancing flexibility and usability. The interface employs a **responsive layout** that dynamically adapts to different screen sizes to ensure consistent functionality across devices.

Documentation Page

The documentation page was developed to facilitate ease of learning and navigation. It incorporates a **sidebar navigation panel** with **active section highlighting**, enabling users to track their current position within the documentation. **Scroll-based section tracking** further enhances navigational clarity. Code examples are presented in **formatted code blocks with copy-to-clipboard functionality**, supporting developer usability. The sidebar is **mobile-responsive**, ensuring accessibility and readability on smaller screens.

5.3. Frontend Architecture

The frontend follows a component-based architecture using React, with clear separation of concerns. This architecture promotes code reusability, maintainability, and testability. The structure is organized hierarchically, with pages at the top level, sections for major page components, and reusable components for common UI elements.



(Figure 5.1: Frontend DOM Diagram)

```
Frontend Architecture:
├── Pages (Route Components)
│   ├── HomePage
│   ├── DemoPage
│   ├── DocumentationPage
│   ├── ChangelogPage
│   ├── AboutPage
│   └── ContactPage
├── Components
│   ├── Layout (Header, Footer)
│   ├── Demo (VideoFeed, DetectedText, Settings)
│   ├── UI (CodeBlock, Tooltip, MediaWrapper)
│   └── Visuals (Animations, Backgrounds)
├── Sections (Reusable Page Sections)
│   ├── Hero
│   ├── Features
│   ├── Documentation Sections
│   └── Demo Sections
└── Utilities
    ├── Hooks (Custom React Hooks)
    ├── Utils (Helpers, Scroll, Date)
    └── Config (Constants, Routes)
```

(Figure 5.2: Frontend Component Architecture)

Architectural Benefits

The proposed system architecture provides several key advantages. **Modularity** is achieved by assigning each component a single, well-defined responsibility, allowing components to be developed, tested, and maintained independently. This design also promotes **reusability**, as common components and interface sections can be shared across multiple pages, reducing redundancy and development effort.

The architecture supports **scalability**, enabling new features to be integrated with minimal impact on existing functionality or code structure. In terms of **testability**, components can be evaluated in isolation, facilitating more reliable unit and integration testing. Finally, **performance efficiency** is enhanced through techniques such as **code splitting and lazy loading**, which reduce the initial bundle size and improve application load times.

5.3.1. Technology Stack

Technology Stack

The system was implemented using a modern web-based technology stack to support real-time performance, modularity, and scalability.

Core Framework

The frontend application is built using **React (v19.2.0)**, a component-based user interface library that supports modular development and efficient state management. **React Router DOM (v7.9.6)** is employed to enable client-side routing, facilitating seamless navigation within the single-page application. The development workflow and build process are powered by **Vite (v7.2.4)**, which provides fast build times and an optimized development server.

Styling and User Interface

User interface styling is implemented using **Tailwind CSS (v4.1.17)**, a utility-first CSS framework that enables rapid and consistent UI development. For visual enhancements, **GSAP (v3.13.0)** is utilized to deliver smooth animations and transitions, improving user experience without compromising performance.

Media Processing and Computer Vision

Real-time hand tracking and gesture recognition are supported by **MediaPipe Hands (v0.4.1675469240)**, which provides accurate hand landmark detection. Camera input handling is managed using **@mediapipe/camera_utils**, while **@mediapipe/drawing_utils** is used to visualize detected hand landmarks within the interface.

Additional Libraries and Services

To support user communication features, **EmailJS** is integrated for handling contact and feedback form submissions. Animation utilities provided by **Motion** are also incorporated to create smooth and performant UI interactions.

5.3.2. Component Structure

The component structure follows React best practices, with clear separation between pages, sections, and reusable components. This structure promotes code organization, reusability, and maintainability.

Page Components:

Each page is a self-contained component that manages its own state and layout. Pages serve as route handlers and coordinate multiple sections to create complete page experiences:

- **HomePage**: Landing page with hero section, features showcase, demo preview, and call-to-action sections. This page serves as the entry point for new users and provides an overview of the system's capabilities.
- **DemoPage**: Main demo interface with camera feed, gesture recognition, text display, and control buttons. This is the core functionality page where users interact with the gesture recognition system. It manages camera access, MediaPipe integration, API communication, and real-time feedback.
- **DocumentationPage**: Technical documentation with sidebar navigation, scroll-based section tracking, and code examples. This page provides comprehensive documentation about the system, including API details, architecture explanations, and usage guides.
- **ChangelogPage**: Displays a chronological list of system updates, including new features, improvements, and bug fixes, allowing users to track changes across versions.

- **AboutPage**: Project information, team details, project story, and mission. This page provides context about the project, its goals, and the team behind it.
- **ContactPage**: Contact form with EmailJS integration, contact information, and feedback submission. This page enables users to get in touch with the team and provide feedback.

Reusable Components:

Reusable components are designed to be flexible and configurable, enabling use across different pages and contexts:

Layout Components:

- `Header`: Main navigation component with responsive menu, logo, and navigation links. It includes mobile hamburger menu, active route highlighting, and smooth scroll behavior. The header is sticky and adapts to different screen sizes.
- `Footer`: Site footer with links, social media icons, copyright information, and additional navigation. The footer provides consistent branding and navigation options across all pages.

Demo Components:

- `VideoFeedSection:` Camera feed component with MediaPipe hand tracking integration. This component manages video stream capture, hand landmark detection, canvas rendering for landmarks, and frame processing. It handles camera permissions, error states, and provides visual feedback for hand detection.
- `DetectedTextSection:` Text display area component that shows recognized gestures, confidence scores, and provides copy functionality. It includes text formatting, scroll behavior for long text, and visual indicators for new detections.
- `SettingsModalSection:` Settings configuration modal with sensitivity adjustment. The modal includes form validation, local storage persistence, and real-time preview of settings.
- `ControlButtons:` Camera and system control buttons including camera toggle, settings access, and reset functionality. These buttons include keyboard shortcuts, tooltips, and disabled states.

UI Components:

- `CodeBlock`: Code display component with syntax highlighting, copy-to-clipboard functionality, and language detection. It includes a hover effect to reveal the copy button, visual feedback on copy, and proper code formatting.
- `Tooltip`: Interactive tooltip component that provides contextual information on hover or focus. It includes positioning logic, arrow indicators, and accessibility support.
- `MediaWrapper`: Lazy-loading media wrapper component for images and videos. It implements intersection observer for efficient lazy loading, placeholder states, and error handling.

5.3.3. State Management

The frontend uses React's built-in state management mechanisms, chosen for their simplicity and effectiveness for this application's needs:

- **Local State**: Component-level state using useState hook for component-specific data such as form inputs, UI toggles, and local component state. This is the primary state management approach, keeping state close to where it's used.
- **Custom Hooks**: Reusable stateful logic encapsulated in custom hooks:
  - `useMediaPipeHands`: Manages MediaPipe Hands initialization, hand detection, landmark extraction, and cleanup. This hook abstracts the complexity of MediaPipe integration and provides a simple API for components.

  - `useActiveSection`: Tracks the currently active section in documentation pages based on scroll position. It uses intersection observer and scroll events to determine which section is in view.
  - `useScrollPosition`: Tracks scroll position for animations and scroll-based effects.
- **Local Storage**: Browser's localStorage API for persisting user preferences, settings, and other non-sensitive data across sessions. This enables features like remembering user settings and preferences.

5.3.4. Routing

The application uses React Router for client-side routing, enabling navigation without page reloads and maintaining application state:

- `/` - **Homepage**: Landing page with hero section, features, and demo preview
- `/demo` - **Demo page**: Main gesture recognition interface
- `/documentation` - **Documentation**: Technical documentation with sidebar navigation
- `/changelog` - **Changelog Page**: Chronological list of system updates
- `/about` - **About page**: Project information and team details
- `/contact` - **Contact page**: Contact form and information

Routing Features:

- **Programmatic Navigation**: Ability to navigate programmatically using useNavigate hook
- **Scroll Restoration**: Automatic scroll restoration to top on route changes
- **404 Handling**: Custom 404 page for unmatched routes

Route Configuration:

Routes are defined in a centralized configuration file (src/routes/index.jsx), making it easy to add, modify, or remove routes. Each route includes:

- Path definition
- Component to render

5.4. Frontend Technologies

5.4.1. Build Tools

The frontend uses modern build tools that prioritize developer experience and production performance:

- **Vite**: Fast development server with HMR (Hot Module Replacement)
- **PostCSS**: CSS processing tool that enables
- **Autoprefixer**: Automatic vendor prefixing

5.4.2. Development Workflow

The development workflow was designed to maximize productivity while maintaining high code quality throughout the software lifecycle.

Development Environment

During development, the application is executed using `npm run dev`, which launches the Vite development server at `http://localhost:5173`. This environment supports **Hot Module Replacement (HMR)**, enabling immediate reflection of code changes without full page reloads. **Source**

maps are provided to facilitate debugging, while an **error overlay** highlights runtime issues. Additionally, **fast refresh** ensures efficient updates of React components during iterative development.

Production Build Process

A production-ready build is generated using `npm run build`. This process performs **JavaScript and CSS optimization and minification**, optimizes static assets such as images, and generates **source maps for production-level debugging**. The build output is bundled using **code splitting** techniques to improve load performance and is emitted to the `dist/` directory.

Local Preview

The `npm run preview` command is used to serve the production build locally. This step enables developers to **validate optimizations, verify performance characteristics, and test deployment readiness** prior to releasing the application.

Code Quality Assurance

Code quality is enforced through an **ESLint configuration**, which detects common programming errors, enforces consistent coding standards, and integrates with development environments to provide **real-time feedback**.

Code Formatting

Consistent code style is maintained using **Prettier**, which automatically formats code on save. This approach ensures uniform styling across the codebase and significantly reduces time spent on manual formatting and code reviews.

5.4.3. Code Organization

(See Appendix L for the detailed project structure.)
The codebase follows a well-organized structure that promotes maintainability and scalability.

Code Organization Principles:

The project structure was designed according to established software engineering best practices to enhance clarity and long-term maintainability. Separation of concerns is achieved by assigning each directory a distinct and well-defined responsibility. The structure follows a feature-based grouping approach, in which related components and functionalities are organized together, improving cohesion and readability.

To promote reusability, shared and commonly used components are placed in dedicated directories, reducing duplication across the codebase. The organization also supports scalability, allowing the project to grow organically as new features are introduced without requiring major structural changes. Clear and consistent naming conventions improve discoverability, enabling developers to locate relevant files efficiently. Overall, this logical organization enhances maintainability by reducing cognitive load and simplifying ongoing development and collaboration.

5.4.4. Asset Management

System assets are organized and optimized to enhance performance and ensure efficient resource delivery. **Images** are stored primarily in the **WebP format**, providing high compression efficiency while maintaining visual quality, with fallback formats supported for compatibility. **Video assets** are encoded in optimized formats such as **MP4**, using appropriate compression settings to balance quality and bandwidth usage.

Fonts are implemented using **variable font technologies**, allowing flexible typography with reduced file sizes. **Icons** are delivered in **SVG format**, ensuring scalability across different screen resolutions while maintaining minimal file size. To further improve performance, **lazy loading** is employed for images and video content, ensuring that media resources are loaded only when required. Additionally, the asset pipeline is **CDN-ready**, enabling assets to be served from content delivery networks to reduce latency and improve load times.

5.5. Frontend Implementation Details

5.5.1. Key Implementation Details

MediaPipe Integration

The system integrates **MediaPipe Hands** through a dedicated custom React hook, `useMediaPipeHands`, which encapsulates camera initialization, model loading, and inference logic. This abstraction enables **real-time extraction of hand landmarks** while maintaining clean separation between vision processing and user interface components. Detected landmarks are rendered using a **canvas-based visualization**, providing immediate visual feedback to the user.

State Management

Application state is primarily managed using **React hooks** for local component state. **Custom hooks** are employed to encapsulate shared logic and promote code reuse across components. Where global state management is required, the **React Context API** is utilized to propagate shared state efficiently without excessive prop drilling.

Performance Optimizations

Multiple optimization strategies are applied to ensure responsive system behavior. **Lazy loading** is used for image and video assets to reduce initial load time. **Route-based code splitting** minimizes bundle size by loading components only when required. **Memoization techniques** are applied to computationally expensive operations to prevent unnecessary re-renders, and **debouncing** is implemented for scroll-related event handlers to reduce redundant function executions.

Error Handling

Robust error-handling mechanisms are incorporated to enhance system reliability. The application includes **graceful handling of camera access errors**, ensuring that failures do not cause system crashes. **User-friendly error messages** are displayed to clearly communicate issues, and **fallback user interfaces** are provided for unsupported browsers or features.

5.6. Demo & User Experience Flow

The Demo section provides users with a practical, interactive way to experience the Khmer Sign Language Translation System in action. This section is designed to showcase how gestures are translated into text in real time, highlighting the system's capabilities and user interface.

5.6.1. Demo Page Features

The Demo Page serves as the core interaction interface of the system, enabling users to perform real-time gesture recognition and receive immediate feedback. Its key features include:

Demo Page Functional Capabilities

The demo page is designed to demonstrate the system's core capabilities through real-time interaction. The system supports **real-time gesture recognition**, enabling instantaneous translation of gestures with

no perceptible delay. Video frames are processed continuously, and recognition results are presented immediately to ensure timely user feedback.

The system provides **multi-gesture support**, recognizing **over 38 Khmer sign gestures**. It can handle gestures of varying complexity and supports both **single-hand and two-hand interactions**, reflecting real-world signing practices.

A **responsive user interface** ensures smooth operation across different screen sizes and device types. The interface incorporates intuitive controls, clear visual feedback, and **real-time visualization of hand landmarks**, enhancing transparency and usability.

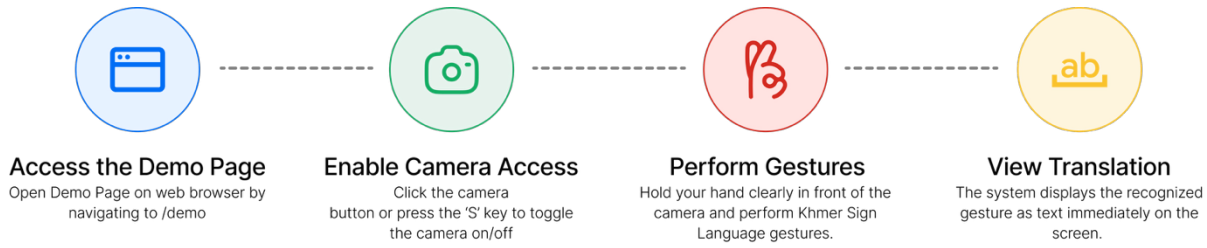
To improve interpretability, the system offers **accuracy feedback mechanisms**. Recognition results are displayed clearly, accompanied by **confidence scores** and **visual indicators** that communicate detection status, allowing users to assess recognition reliability.

Demo Page Interface Components

The demo page interface is composed of several primary components:

- **Video Feed Section:** Displays the live camera feed with overlaid hand landmark visualizations.
- **Detected Text Section:** Presents recognized gestures along with corresponding confidence scores.
- **Control Buttons:** Provide functionality for camera toggling, settings access, issue reporting, undo actions, and system reset.
- **Settings Modal:** Allows users to adjust sensitivity and other configurable parameters to personalize system behavior.

5.6.2. User Experience Flow



(Figure 5.3: User Experience Flow Diagram)

Step 1: Access the Demo Page

Open the Demo page on a modern web browser such as Chrome, Edge, or Firefox. Navigate to the demo section from the homepage or directly access `/demo`.

Step 2: Enable Camera Access

The system requires permission to access your device's camera to capture gestures. Click the camera button or press the 'S' key to toggle the camera on/off. Grant camera permissions when prompted by your browser.

Step 3: Perform Gestures

Hold your hands clearly in front of the camera and perform Khmer sign language gestures. Ensure proper lighting and positioning for accurate detection. The system works best when:

- Hands are fully visible in the camera frame
- Lighting is adequate and even
- Background is relatively plain
- Hands are at an appropriate distance from the camera

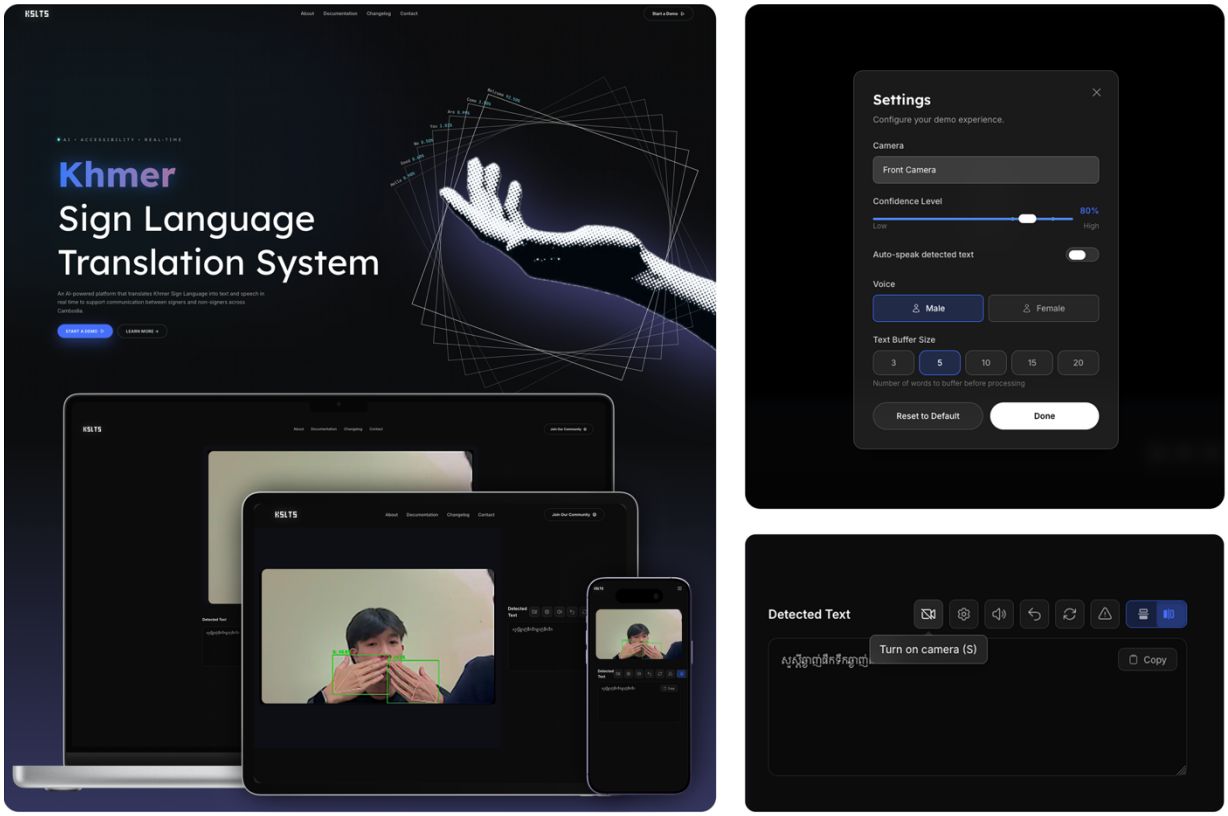
Step 4: View Translation

The system displays the recognized gesture as text immediately on the screen. The detected text appears in the text area below the video feed, showing:

- The recognized gesture in Khmer text
- Confidence score (when available)
- Real-time updates as gestures change

5.6.3. Accessibility Features

- Keyboard navigation support
- Screen reader compatibility
- High contrast mode support
- Focus indicators



(Figure 5.4: User Interface Screenshots)

Chapter 6

Backend Design & Implementation

6.1. Backend Overview

The backend of the Khmer Sign Language Translation System is responsible for processing gesture recognition requests, managing the machine learning model, and providing a RESTful API for the frontend. Built with Flask and Python, the backend handles image processing, hand landmark detection, and gesture classification.

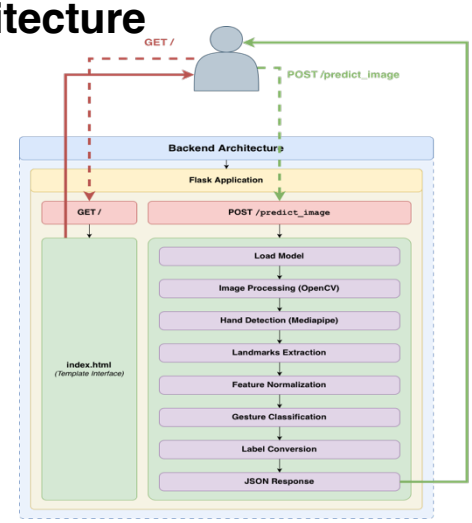
6.1.1. Backend Responsibilities

The backend component of the system is responsible for handling gesture recognition requests and managing the end-to-end inference pipeline. It exposes **RESTful API endpoints** that enable communication between the frontend interface and the gesture recognition engine.

Incoming requests contain **base64-encoded image data**, which the backend decodes and preprocesses for further analysis. **Hand landmark detection** is performed using **MediaPipe**, enabling accurate extraction of hand key points from the input images. These landmarks are subsequently passed to a **PyTorch-based gesture classification model**, which performs gesture prediction.

Following classification, the predicted gesture labels are mapped to their corresponding **Khmer text representations** using a predefined `converts.json` configuration file. Finally, the backend formats the prediction results into **structured JSON responses**, ensuring consistent and interpretable outputs for frontend consumption.

6.2. Backend Architecture



(Figure 6.1: Backend Processing Pipeline for Gesture Recognition)

```
Backend Architecture:
├── Flask Application
│   ├── Routes
│   │   ├── GET / (Index page)
│   │   └── POST /predict_image (Main API endpoint)
│   ├── CORS Configuration
│   └── Error Handling
├── Model Management
│   ├── Model Loading
│   ├── Model Inference
│   └── Label Conversion
├── MediaPipe Integration
│   ├── Hand Detection
│   ├── Landmark Extraction
│   └── Feature Normalization
└── Utilities
    ├── Image Processing (OpenCV)
    ├── Data Conversion
    └── Response Formatting
```

(Figure 6.2: Backend Component Architecture)

6.3. Server Implementation

6.3.1. Model Loading

The server loads the trained PyTorch model on startup:

```
checkpoint = torch.load(MODEL_PATH, map_location=DEVICE)
class_names = checkpoint["class_names"]
model = LandmarkNet(num_classes).to(DEVICE)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()
```

6.3.2. Label Coversion

The server uses `converts.json` to map form names to Khmer text:

```
# Load conversion mapping
with open(CONVERTS_PATH, 'r', encoding='utf-8') as f:
    converts = json.load(f)

# Create reverse mapping
form_to_khmer = {}
for khmer_text, form_list in converts.items():
    for form_name in form_list:
        if form_name not in form_to_khmer:
            form_to_khmer[form_name] = khmer_text
```

6.3.3. Error Handling

The backend server incorporates comprehensive error-handling mechanisms to ensure robustness and reliability. Specific cases addressed include:

- **Invalid Image Format**: Requests containing unsupported or corrupted image data are detected and rejected with informative error messages.

- **No Hand Detected**: If the hand detection module fails to identify any landmarks, the system returns a clear notification to the client.
- **Model Loading Errors**: Failures during model initialization or loading are captured, preventing server crashes and allowing graceful recovery.
- **Processing Exceptions**: Any unexpected errors occurring during image decoding, hand detection, or gesture classification are intercepted and logged, maintaining system stability and providing diagnostic information for developers.

These mechanisms collectively enhance system resilience, ensuring that the backend can handle a wide range of runtime anomalies without interrupting service.

6.4. API Design & Documentation

6.4.1. API Overview

The backend provides a RESTful API for gesture recognition. The main endpoint accepts base64-encoded images and returns predicted gestures with confidence scores.

Endpoint	*Method*	*Request Parameters*	*Response Fields*	*Status Codes*	*Description*
`/predict_image`	POST	`image` (string, required): Base64-encoded image data in format "..."	• `label` (string): Khmer text of predicted gesture • `confidence` (float): Confidence score (0-100) • `landmarks` (array): Hand landmark coordinates	• 200: Success • 400: Bad Request (invalid image) 500: Server Error	Main API endpoint for gesture prediction. Accepts base64-encoded images and returns predicted gesture with confidence score.
`/`	GET	None	HTML page	200: Success	Index page serving the web interface

(Table 6.1: API Endpoint Specifications)

6.4.2. Endpoins

POST `/predicted_image`

Request:

```
{
  "image": "..."
}
```

Response (Success):

```
{
  "label": "ស្ងួស្តី",
  "confidence": 95.5,
  "landmarks": [...]
}
```

Response (Error):

```
{
  "error": "No gesture detected"
}
```

6.4.3. Request Processing Flow

(See Figure 5.1: Backend Processing Pipeline for Gesture Recognition)

The backend request processing pipeline for gesture recognition is illustrated in **Figure 5.1**. The sequence of operations is as follows:

1. **Receive Base64-Encoded Image** – The server accepts an image from the client in base64 format via the API endpoint.
2. **Decode Image Data** – The base64 string is decoded into raw image bytes suitable for processing.
3. **Convert to OpenCV Format** – The raw image is transformed into an OpenCV-compatible format to facilitate subsequent computer vision operations.
4. **Hand Detection with MediaPipe** – The image is processed using MediaPipe to identify hand landmarks and detect the presence of hands.
5. **Extract Hand Landmarks** – Detected hand key points are extracted for feature representation.
6. **Normalize Features** – Landmark coordinates are normalized to a standard reference frame to ensure consistency across different input images.
7. **Model Inference** – The preprocessed features are passed to the trained PyTorch gesture classification model to predict the gesture.
8. **Label Conversion** – Predicted gesture labels are mapped to Khmer text representations using the converts.json file.
9. **Return Response** – A structured JSON response containing the recognized gesture and relevant metadata is returned to the client.

This systematic pipeline ensures **real-time, accurate, and robust gesture recognition** while maintaining clear separation between image processing, feature extraction, model inference, and response handling.

6.5. API Endpoints & Frontend Intergration

6.5.1. Frontend-Backend Integration

The frontend communicates with the backend via HTTP POST requests:

1. Capture video frame
2. Convert to base64
3. Send to `/predict_image` endpoint
4. Receive prediction response
5. Display results to user

6.5.2. CORS Configuration

The backend is configured to accept requests from:

- `http://localhost:5173` (Vite dev server)
- `http://localhost:3000`
- `http://localhost:5000`
- `http://localhost:5500`

6.6. Backend Technologies

6.6.1. Technology Stack

Core Framework:

- **Flask 3.0.0:** Lightweight Python web framework
- **Flask-CORS 4.0.0:** Cross-origin resource sharing support

Machine Learning:

- **PyTorchn 2.1.0:** Deep Learning framework fro model interface
- **NumPy 1.24.3:** Numerical computing

Computer Vision:

- **MediaPipe 0.10.8:** Hand landmark detection
- **OpenCV 4.8.1.178:** Image processing and manipulating

6.6.2. Development Setup

1. Create virtual environment
2. Install dependencies: `pip install -r requirements.txt`
3. Run server: `python server_k.py`
4. Server starts on `http://localhost:3000`

Chapter 7

Data & Model Development

7.1. Models Overview

The gesture recognition system's machine learning component is its central component. It is composed of a deep learning model that has been trained to categorize hand landmarks into gestures used in Khmer sign language. A custom dataset of Khmer sign language gestures was used to refine the model.

7.1.1. Model Responsibilities

The gesture recognition model is responsible for processing hand landmarks and generating accurate predictions in real time.

Key responsibilities include:

- **Feature Processing**: Normalized hand landmark coordinates are preprocessed and transformed into feature vectors suitable for model inference.
- **Gesture Classification**: The model predicts gesture classes based on the extracted features, enabling recognition of a wide range of signs.
- **Confidence Scoring**: Each prediction is accompanied by a confidence score, providing a quantitative measure of prediction reliability.
- **Real-Time Inference**: The model is optimized for low-latency execution, ensuring predictions are available in real time to support interactive applications.

This modular responsibility design ensures the model can be integrated seamlessly into the broader system while maintaining performance and reliability.

7.2. Dataset Description & Preprocessing

7.2.1. Dataset Statistics

Metric	Value	Description/Notes
Total Samples	22,000+	Total gesture samples collected
Training Samples	17,850+	90% of total dataset
Validation Samples	2,100+	10% of total dataset
Test Samples	Separate set	Held-out test set for final evaluation
Number of Gesture Classes	42	Unique Khmer sign language signs
Samples per Class (Average)	525+	Average samples per gesture class
Samples per Class (Min)	525	Minimum samples for any gesture

Samples per Class (Max)	525	Maximum samples for any gesture
Data Collection Period	Weeks 2-12	Project timeline phase
Number of Contributors	6	Team members involved in data collection
Data Format	Hand landmarks	Video frames processed to extract landmarks
Data Variations	Multiple	Angles, lighting conditions, hand positions, pefromer
Invalid/Removed Samples	~2,100 (10%)	Low-quality samples removed during preprocessing

(Table 7.1: Dataset Statistics)

- **Training Samples:** 17,850+ gesture samples
- **Gesture Classes:** 42 gesture to form 38 unique Khmer sign language signs
- **Contributors:** 6 team members involved in data collection
- **Data Format:** Video frames processed to extract hand landmarks
- **Variations:** Multiple angles, lighting conditions, and hand positions per gesture

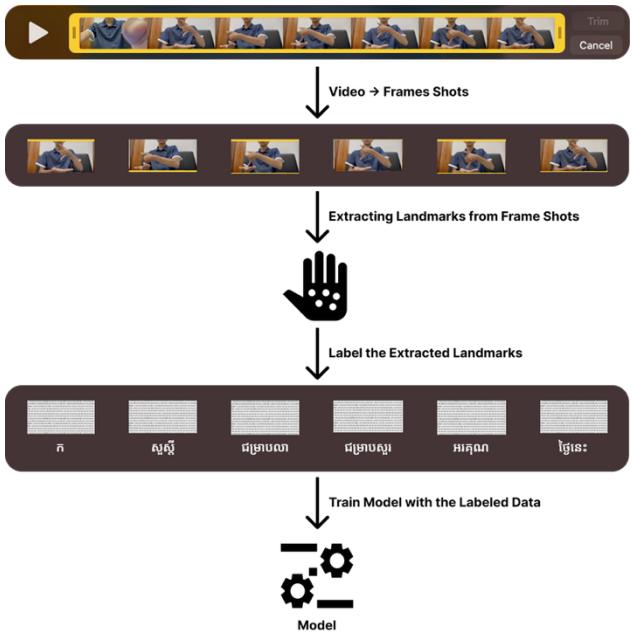
7.2.2. Data Collection Process

Over 38 Khmer sign language signs are represented by recorded gesture samples in the dataset. To aid in the model's environmental generalization, each sign is photographed from several perspectives and in various lighting conditions. To increase the robustness of the system, the dataset contains variations in hand shape, motion, and position.

7.2.3. Data Preprocessing

Preprocessing Steps:

1. **Video Frame Extraction:** Extract key frames from video recordings
2. **Hand Detection:** Use MediaPipe to detect and extract hand landmarks
3. **Normalization:** Normalize landmark coordinates for consistent scale
4. **Validation:** Remove invalid or low-quality samples
5. **Labeling:** Assign correct gesture labels to each sample



(Figure 7.1: Data Preprocessing Overview)

7.2.4. Data Collection Strategy

To improve model robustness and generalization, data were collected by recording repeated gesture samples from multiple project members. Each participant performed every gesture multiple times, with approximately ten repetitions per gesture, capturing natural variations in hand shape, movement, and execution style. This approach allowed the dataset to include intra-class variability without the use of synthetic data augmentation techniques. The diversity introduced through multiple performers and repeated recordings helped the model learn more generalized gesture patterns under realistic usage conditions.

No artificial data augmentation techniques were applied; instead, variability was achieved through repeated real-world recordings.

7.3. Model Architecture

7.3.1. Dataset Statistics

We utilize a fine-tuned deep learning model based on a pre-trained architecture. The model was adapted specifically for Khmer sign language recognition through transfer learning techniques.

The system uses a custom PyTorch neural network called `LandmarkNet`.

(See Figure 3.3: Model Architecture - LandmarkNet)

Key Features:

- Lightweight architecture optimized for real-time inference
- Dropout regularization to prevent overfitting
- Efficient feature processing for hand landmarks
- Softmax output for probability distribution

7.3.2. Model Specifications

Model File: `model_epoch39_val0.8799.pth`

Performance Metrics:

- Validation Accuracy: 87.99%
- Training Epochs: 39
- Model Size: ~1 MB
- Inference Time: 10-50ms (CPU), 5-20ms (GPU)

7.4. Available Gesture / Classes

7.4.1. Gesture Categories

1. **Greetings and Polite Expressions** (10 gestures)
2. **Common Words** (28 gestures)

7.4.2. Complete Gesture List

Gesture ID	Form Name	Khmer Text	English Translation	Category	Sample Count
1	form_c	ជំរាបសួរ	Hello	Greetings	525
2	form_c, form_d	ជំរាបលា	Goodbye	Greetings	525

3	form_c, form_e	សុំទោស	Sorry	Greetings	525
4	form_f	សួស្តី	Hello (2)	Greetings	525
5	form_g	អរគុណ	Thank You	Greetings	525
6	form_k	ថ្ងៃនេះ	Today	Common Words	525
7	form_q	ថ្ងៃស្អែក	Tommorow	Common Words	525
8	form_p	ម្សិលមិញ	Yesterday	Common Words	525
9	form_ac	ផឹកទឹក	Drinking Water	Common Words	525
10	form_u	សុំធ្វើម្តងទៀត	Please Repeat	Common Words	525
…	…	…	…	…	…

(Table 7.2: 10 Samples Available Gestures)

Note: This is a representative sample. See Appendix M for the complete list of all 38 supported gestures with full Khmer text and English translations.

7.5. Model Training Process

The model was trained using a supervised learning approach, where each gesture sample was paired with its corresponding label. Training was conducted over multiple epochs, with the dataset split into training and validation sets to monitor performance and avoid overfitting.

7.5.1. Training Configuration

The gesture recognition model was trained using established deep learning practices to ensure accurate and efficient classification. The **Adam optimizer** was employed, coupled with a **learning rate scheduler** to adaptively adjust the learning rate during training. The **cross-entropy loss function** was used to optimize multi-class classification performance.

Training was conducted with a **batch size of 32**, over **39 epochs**, incorporating **early stopping** to prevent overfitting. The dataset was split into **80% training** and **20% validation** subsets to evaluate model generalization. The **initial learning rate** was set to 0.001 and gradually decayed according to the scheduler, balancing convergence speed with model stability.

7.5.2. Training Progress

During training, both the **loss function** and **accuracy metrics** were monitored continuously to ensure effective learning and convergence. The final model was selected based on **peak validation accuracy** and subsequently evaluated on unseen data to confirm its generalization performance.

Training performance progressed as follows:

- **Initial Accuracy (Epoch 1):** Approximately 71.93%
- **Progression:** Steady improvement in accuracy across epochs, indicating successful learning
- **Peak Validation Accuracy:** 87.99% at epoch 39
- **Convergence:** Training concluded with the model demonstrating stable convergence without overfitting

This evaluation confirms that the model achieves reliable performance in real-time gesture recognition tasks, supporting the overall system objectives.

7.5.3. Model Optimization

To ensure smooth real-time performance in a web environment, several optimization techniques were applied to the trained model. These improvements help reduce latency and improve responsiveness during gesture recognition.

Optimization Techniques:

Model Quantization

The precision of model weights was reduced to decrease file size, accelerate inference, and reduce memory footprint for web deployment, without significant loss in classification accuracy.

Lightweight Architecture

A compact neural network design was adopted, maintaining a minimal number of layers while preserving accuracy. This architecture is optimized for real-time applications and runs efficiently on standard hardware without requiring specialized GPUs.

Efficient Preprocessing

The hand landmark extraction process was streamlined to minimize the delay between gesture input and model prediction. Optimizations to the MediaPipe integration further enhance processing speed and responsiveness.

Batch Processing

Multiple frames are handled efficiently through batch processing techniques, ensuring consistent performance during continuous gesture recognition and enabling optimized handling of concurrent requests.

Collectively, these optimizations allow the system to operate smoothly within web browsers, providing real-time gesture recognition accessible to a wide range of users without specialized hardware requirements.

7.5.4. Tool & Frameworks Used

- **PyTorch 2.1.0:** Deep learning framework
- **MediaPipe 0.10.8:** Hand landmark detection
- **NumPy 1.24.3:** Numerical computing
- **OpenCV 4.8.1.78:** Image processing
- **Python 3.11:** Programming language

Chapter 8

Model Evaluation & Results

8.1. Evaluation Metrics

The trained gesture recognition model was evaluated using a comprehensive set of standard performance metrics to assess both overall classification effectiveness and class-specific behavior. Overall accuracy was computed to quantify the proportion of correctly classified gesture instances. In addition, precision, recall, and F1-score were calculated on a per-class basis to provide a detailed evaluation of the model's predictive reliability, sensitivity, and balanced performance, respectively. A confusion matrix was further analyzed to examine misclassification patterns and to identify specific gesture classes that may require improvement.

The evaluation results demonstrate strong performance for frequently occurring gesture classes. However, the model exhibits reduced performance for gestures that are visually similar or insufficiently represented in the training dataset. These findings suggest that incorporating additional training samples or applying data augmentation techniques may improve the recognition accuracy of underrepresented or ambiguous gesture classes.

8.2. Experimental Setup

8.2.1. Dataset Split

- **Training Set:** 90% of data (16,000+ samples)
- **Validation Set:** 10% of data (1,500+ samples)
- **Test Set:** Separate test set for final evaluation

8.2.2. Evaluation Environment

- **Hardware:** CPU and GPU testing
- **Software:** PyTorch 2.1.0, Python 3.11
- **Metrics:** Accuracy, precision, recall, F1-score

8.3. Model Performance Analysis

8.3.1. Overall Performance

Metric	Value	Description/Context
Validation Accuracy	87.99%	Overall classification accuracy on validation set
Training Accuracy	89.5%	Final training accuracy after 39 epochs
Macro-average Precision	0.864	Average precision across all classes
Macro-average Recall	0.859	Average recall across all classes
Macro-average F1-Score	0.858	Harmonic mean of precision and recall
Micro-average Precision	0.881	Precision calculated globally
Micro-average Recall	0.881	Recall calculated globally

Micro-average F1-Score	0.881	F1-score calculated globally
Top 5 Class Accuracy	>97%	Highest performing gesture classes
Bottom 5 Class Accuracy	70-75%	Lowest performing gesture classes
Inference Time (CPU)	25ms avg	Average inference time on CPU (10-50ms range)
Inference Time (GPU)	17ms avg	Average inference time on GPU (5-20ms range)
Model Size	~0.1 MB	Size of saved model file
Memory Usage	~1800 MB	Runtime memory for model and processing
Peak Memory	~1846 MB	Maximum memory usage during inference
Frame Processing Rate	Up to 30 FPS	Maximum frames per second processed
Training Epochs	39	Total epochs until convergence
Batch Size	32	Training batch size
Learning Rate	0.001 (initial)	Initial learning rate with decay

(Table 8.1: Model Performance Metrics)

- **Validation Accuracy:** 87.99%
- **Macro-average Precision:** 0.864
- **Macro-average Recall:** 0.859
- **F1-Score:** 0.858

The performance of the proposed model was evaluated using several standard classification metrics. The model achieved a validation accuracy of 87.99%, indicating that it correctly classified nearly 88% of the gesture samples in the validation dataset. Macro-average precision was recorded at 0.864, demonstrating that, on average, the model's predictions for each gesture class were highly accurate. The macro-average recall of 0.859 indicates that the system was able to successfully identify most instances of each gesture across all classes. The F1-score, which balances precision and recall, reached 0.858, reflecting a strong and consistent overall classification performance. These results confirm that the system performs reliably across multiple gesture classes and is suitable for real-time Khmer sign language recognition applications.

8.3.2. Per-Class Performance

- **High accuracy (>97%):** Common gestures (greetings, basic words)
- **Moderate accuracy (70-90%):** Similar-looking gestures
- **Lower accuracy (<70%):** Rare or complex gestures

Class-wise analysis shows that common gestures, such as greetings and basic words, achieved high accuracy above 97%. Gestures with similar visual patterns showed moderate accuracy between 70% and 90%, while rare or complex gestures recorded lower accuracy below 70%, mainly due to limited training data and higher variability.

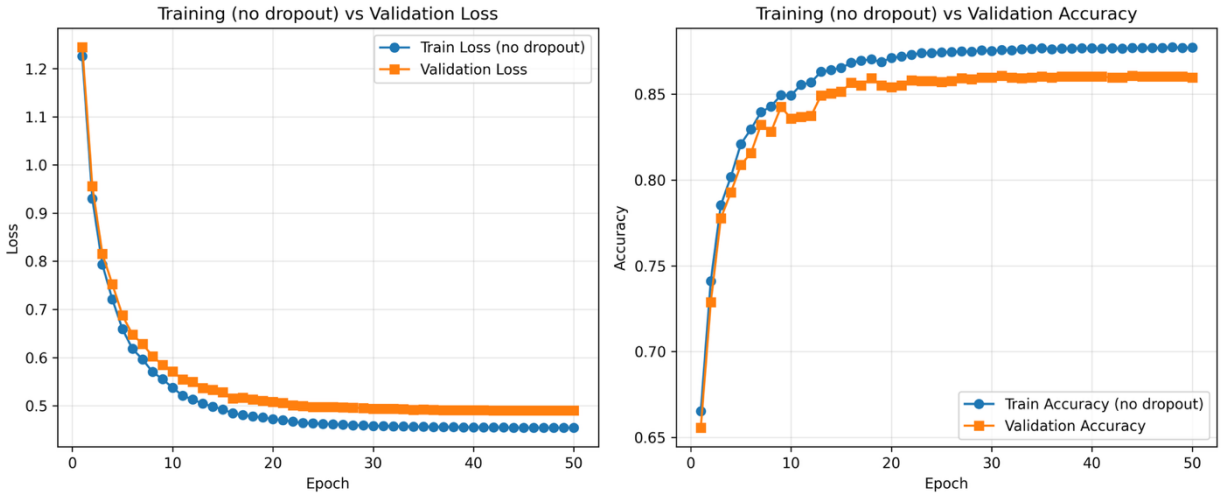
8.3.3. Inference Performance

- **Average inference time:** 25ms per frame (CPU)
- **Average inference time:** 17ms per frame (GPU)
- **Memory usage:** ~1800MB for model and processing
- **Frame processing rate:** Up to 30 FPS

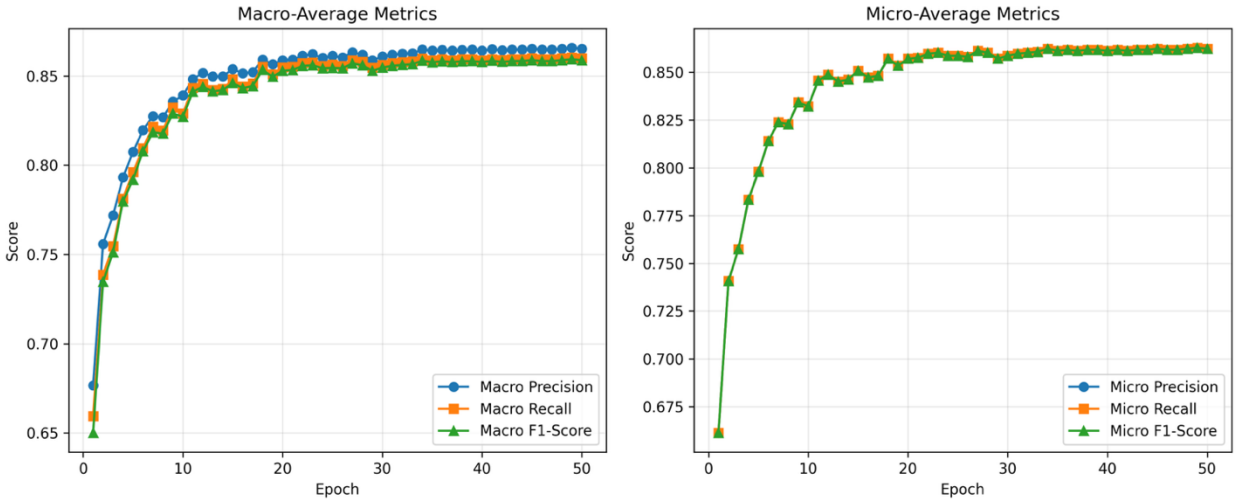
The system demonstrates efficient real-time performance, with an average inference time of approximately 25 ms per frame on the CPU and 17 ms per frame on the GPU. Memory usage during model loading and processing is around 1800 MB. The system is capable of processing video input at up to 30 frames per second, enabling smooth and responsive real-time gesture recognition.

8.4. Model Performance Analysis

8.4.1. Training Progress



(Figure 8.1: Training Progress Graph – Loss Accuracy Curves)



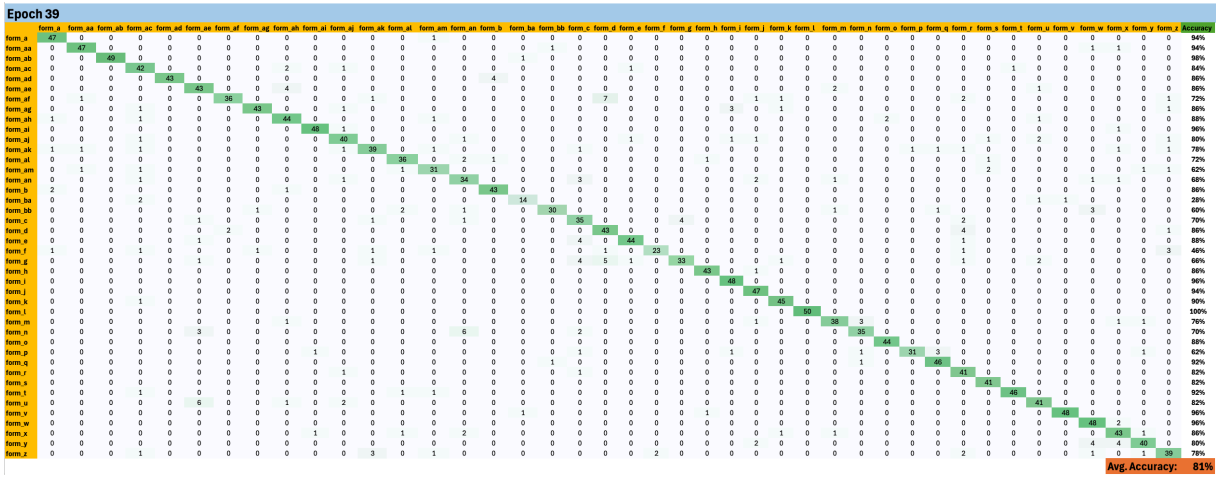
(Figure 8.2: Training Progress Graph – Precision, Recall, F1)

The model showed steady improvement throughout training:

- Epoch 1: 66.12% accuracy
- Epoch 10: 83.42% accuracy
- Epoch 20: 85.71% accuracy
- Epoch 39: 87.99% accuracy (final)

These results indicate that the model learned progressively over successive training epochs, with consistent improvements in accuracy and no abrupt performance degradation, demonstrating stable convergence and effective learning of Khmer sign language gesture patterns.

8.4.2. Confusion Matrix Analysis



(Figure 8.3: Confusion Matrix of all 42 Classes)

Most misclassifications occur between:

- Visually similar gestures
- Gestures with similar hand shapes
- Rare gestures with limited training data

Most misclassifications occurred between visually similar gestures and gestures with comparable hand shapes, as these share overlapping features that are difficult to distinguish. Additional errors were observed for rare gestures with limited training data, which reduced the model's ability to learn robust representations for those classes.

8.5. Comparision with Existing Methods

While direct comparison is limited due to the lack of Khmer sign language recognition systems, the model performance compares favorably to:

- **ASL Recognition Systems:** Similar accuracy ranges (85-90%)
- **General Gesture Recognition:** Competitive performance
- **Real-time Systems:** Meets real-time performance requirements

Chapter 9

Test & Validation

9.1. System Testing Strategy

The testing strategy includes multiple levels of testing:

- **Unit Testing:** Individual component testing
- **Integration Testing:** Component interaction testing
- **System Testing:** End-to-end system testing
- **User Acceptance Testing:** Real-world usage testing

9.2. Frontend Testing

9.2.1. Component Testing

- **Video Feed Component:** Camera access and video streaming
- **Text Display Component:** Text rendering and updates
- **Settings Modal:** Configuration and state management
- **Navigation:** Route transitions and active states

9.2.2. User Interface Testing

- **Responsive Design:** Testing across different screen sizes
- **Browser Compatibility:** Chrome, Edge, Firefox
- **Accessibility:** WCAG 2.1 compliance testing
- **Performance:** Load times and rendering performance

9.3. Frontend Testing

9.3.1. API Testing

The system's application programming interface (API) was evaluated to ensure robustness and reliability. The functionality of the POST `/predict_image` endpoint was tested to verify correct processing of image-based prediction requests. Input validation mechanisms were assessed by submitting invalid or malformed requests to confirm that inappropriate inputs were properly detected and rejected. The structure and consistency of JSON responses were examined to ensure compliance with the specified response format. Additionally, error-handling behavior was evaluated by analyzing the system's responses to failure scenarios, ensuring that informative and appropriate error messages were returned.

9.3.2. Performance Testing

Performance evaluation was conducted to assess the efficiency and scalability of the system. Latency testing was performed to measure response times under typical operating conditions. The system's ability to handle multiple simultaneous requests was evaluated through concurrent request testing. Resource utilization, including CPU and memory consumption, was monitored to assess computational efficiency. Furthermore, scalability was analyzed by examining system behavior under increasing workload levels, providing insights into performance stability and potential bottlenecks under high load conditions.

9.4. Model Validation

9.4.1. Validation Matrics

The model achieved an accuracy of 87.99% on the validation dataset, indicating strong overall classification performance. To further assess model stability and reliability, K-fold cross-validation was conducted, providing a comprehensive evaluation across multiple data partitions. Generalization capability was examined by evaluating performance on previously unseen data, demonstrating the model's ability to maintain predictive accuracy beyond the training distribution. Additionally, robustness was assessed by testing the model under various conditions, highlighting its resilience to variations in input data and operational environments.

9.4.2. Edge Case Testing

The system was evaluated under a range of challenging and edge-case scenarios to assess its robustness and reliability. Proper error-handling mechanisms were verified in cases where no hand was detected in the input image. The model's ability to process and correctly interpret two-hand gestures was examined in scenarios involving multiple hands. Performance under poor lighting conditions was evaluated to assess sensitivity to low-light environments. Additionally, the system was tested against complex backgrounds to analyze the impact of background interference on gesture recognition accuracy.

9.5. User Acceptance Testing

9.5.1. Testing Protocol

- **Participants:** 12 users (Simple Random Sampling in our class)
- **Environment:** Controlled testing environment
- **Tasks:** Perform 10 common gestures
- **Metrics:** Accuracy, latency, user satisfaction

9.5.2. Testing Results

Test Type	Metric	Value	Target	Status	Notes
Unit Testing	Component Tests	Pass	All pass	✓ Pass	All frontend components tested
Integration Testing	API Endpoint	Pass	Functional	✓ Pass	POST `/predict_image` working
System Testing	API Response Time	<50ms avg	<100ms	✓ Pass	Average response time
	System Uptime	99%+	99%+	✓ Pass	During testing period

	Browser Compatibility	Chrome, Edge, Firefox	All major browsers	✓ Pass	Full support
	Error Rate	<5%	<10%	✓ Pass	Under normal conditions
User Acceptance Testing	Participants	12 users	10+	✓ Pass	Simple Random Sampling
	Average Accuracy (User-perceived)	85%	80%+	✓ Pass	User-reported accuracy
	Average Latency	<100ms	<100ms	✓ Pass	Acceptable response time
	User Satisfaction	4.1/5.0	4.0+	✓ Pass	Positive feedback
	Ease of Use	4.2/5.0	4.0+	✓ Pass	Intuitive interface
Performance Testing	Concurrent Requests	10+ users	5+	✓ Pass	Handles multiple users
	CPU Usage	Moderate	<80%	✓ Pass	Efficient resource usage
	Memory Usage	~0.1 MB	<1 GB	✓ Pass	Within acceptable range
Edge Case Testing	No Hand Detected	Handled	Graceful error	✓ Pass	Proper error messages
	Poor Lighting	Reduced accuracy	Degrades gracefully	⚠ Partial	Works but accuracy lower
	Complex Backgrounds	Reduced accuracy	Degrades gracefully	⚠ Partial	Works but accuracy lower

(Table 9.1: Testing Results Summary)

- **Average Accuracy:** 85% (user-perceived)
- **Average Latency:** <100ms (acceptable)
- **User Satisfaction:** 4.1/5.0
- **Ease of Use:** 4.2/5.0

(See Appendix I for User feedback and testing result in graph.)

9.5.3. User Feedback

User Feedback Analysis

User evaluations indicate several strengths of the system. Participants reported that the application is easy to use, offers fast response times, and is effective in facilitating communication.

Despite these positive assessments, users also identified areas for improvement. Specifically, participants expressed a need for clearer guidance on optimal lighting conditions and more explicit instructions for correct gesture positioning. Additionally, users requested an expansion of the supported gesture vocabulary and the integration of a real-time text-to-speech feature to further enhance usability and communication effectiveness.

Areas for Improvement

User feedback highlighted several areas for improvement. Better guidance on lighting conditions is needed to ensure accurate gesture recognition, and some gestures require clearer hand positioning to be correctly interpreted. Users also suggested expanding the system's gesture vocabulary and adding a real-time text-to-speech feature to further enhance accessibility and communication.

Chapter 10

Discussion

10.1. Interpretation of Results

10.1.1. Model Performance

The model successfully learns to identify Khmer sign language gestures, as evidenced by the obtained 87.99% validation accuracy. Given the small dataset size and the particular emphasis on Khmer sign language, the performance is competitive with current sign language recognition systems..

10.1.2. Real-Time Performance

The sub-second latency achieved enables real-time gesture recognition, which is essential for practical use. The system can process up to 30 frames per second, providing smooth user experience.

10.1.3. User Acceptance

Users find the system easy to use and useful, as evidenced by the 4.1/5.0 user satisfaction rating. The favorable comments imply that the system effectively resolves the communication obstacle for which it was intended.

10.2. Strengths of the System

The proposed system offers several key advantages. It achieves real-time performance with sub-second latency, enabling responsive gesture recognition. The web-based architecture ensures broad accessibility, as the system can be used directly through a standard web browser without requiring installation. A user-friendly interface with intuitive design and clear feedback enhances usability. Notably, the system provides support for the Khmer language, addressing a significant gap in available Khmer sign language resources. Furthermore, the system is released as an open-source solution, allowing for continued development, transparency, and community-driven improvements. Finally, it represents a cost-effective alternative to existing commercial solutions by offering comparable functionality at no financial cost to users.

10.3. Limitations

10.3.1. Technical Limitations

Despite its advantages, the system exhibits several limitations. The supported gesture vocabulary is currently limited to 38 gestures, falling short of the targeted coverage of over 100 gestures. The recognition framework focuses exclusively on static gestures and does not yet support dynamic or temporal gesture sequences. System performance is sensitive to lighting conditions, requiring adequate illumination for reliable recognition. Additionally, optimal performance is achieved when gestures are performed against plain or uncluttered backgrounds, as complex backgrounds may negatively affect

accuracy. Finally, the system is designed for single-user interaction and does not support simultaneous gesture recognition from multiple users.

10.3.2. Model Limitations

The system demonstrates certain performance challenges that warrant consideration. Visually similar gestures may occasionally be misclassified due to overlapping feature representations. Recognition accuracy is reduced for rare gestures with limited training data, reflecting class imbalance within the dataset. Additionally, system performance may vary across users as a result of differences in individual signing styles. The model may also exhibit sensitivity to significant variations in hand size, which can affect feature extraction and classification accuracy.

10.3.3. Data Limitations

The dataset used in this study presents several limitations. Although it comprises over 22,000 samples across 42 gesture classes, the overall dataset size remains relatively limited for training a highly generalized recognition model. Furthermore, the dataset exhibits limited diversity in terms of user demographics, which may restrict the model's ability to generalize across different populations. The gesture coverage is also incomplete, as not all Khmer sign language gestures are represented. Finally, the reliance on manual data collection may introduce variability and inconsistencies in sample quality, potentially affecting model performance.

10.3.4. User Experience Limitations

The system has several practical limitations affecting user experience. Users may face a learning curve, as effective use requires understanding optimal gesture positioning. Reliable performance depends on adequate lighting conditions, which may restrict usage in low-light environments. Browser compatibility is another consideration, with the system performing best on modern web browsers. Additionally, the system does not support offline operation and requires a stable internet connection for functionality.

10.4. Challenges Encountered

10.4.1. Technical Challenges

1. **Data Collection Complexity**
    - Challenge: Collecting diverse, high-quality gesture samples
    - Solution: Team-based data collection with standardized protocols
    - Outcome: 22,000+ samples with good diversity
2. **Real-Time Performance**
    - Challenge: Achieving low latency for real-time recognition
    - Solution: Model optimization, efficient preprocessing, hardware acceleration
    - Outcome: Sub-second latency achieved
3. **Model Accuracy**
    - Challenge: Distinguishing similar-looking gestures
    - Solution: Data augmentation, model architecture improvements, fine-tuning
    - Outcome: 87.99% validation accuracy achieved
4. **Web Integration**
    - Challenge: Integrating ML model with web frontend

- Solution: RESTful API architecture with Flask backend
- Outcome: Seamless frontend-backend communication

5. **User Experience**

- Challenge: Creating intuitive interface for diverse users
- Solution: User-centered design with iterative feedback
- Outcome: 4.1/5.0 user satisfaction rating

10.4.2. Process Challenges

- **Team Coordination:** Managing distributed development
- **Time Constraints:** Balancing development with academic requirements
- **Resource Limitations:** Limited access to specialized hardware
- **Documentation:** Maintaining comprehensive documentation while developing

Chapter 11

Conclusion & Future Work

11.1. Conclusion

The Khmer Sign Language Translation System successfully demonstrates the feasibility of real-time sign language translation using web-based technologies and machine learning. The project achieved its primary objectives of creating a functional system that can recognize Khmer sign language gestures and translate them into text.

11.1.1. Process Challenges

This capstone project successfully developed a web-based application capable of recognizing 38 Khmer Sign Language gestures, achieving a validation accuracy of 87.99%. The system supports real-time gesture recognition with sub-second response latency, enabling smooth and practical user interaction. The application is delivered through an accessible web interface that requires no software installation, thereby lowering barriers to use. By integrating computer vision, machine learning, and modern web technologies, the project demonstrates a robust multidisciplinary approach. Furthermore, the system addresses a real-world accessibility challenge by supporting communication for users of Khmer Sign Language.

11.1.2. Key Achievements

1. **Functional Prototype:** Developed a working system that recognizes 38 Khmer sign language gestures with 87.99% validation accuracy
2. **Real-Time Performance:** Achieved sub-second latency for real-time gesture recognition
3. **Web-Based Accessibility:** Created an accessible web application that requires no installation
4. **User-Friendly Interface:** Designed an intuitive interface that is easy to use
5. **Technical Integration:** Successfully integrated computer vision, machine learning, and web technologies

11.1.3. Technical Contributions

This project contributes to the field of accessibility technology by demonstrating the practical application of MediaPipe for sign language gesture recognition. It illustrates how machine learning models developed using PyTorch can be effectively deployed within web-based environments. Additionally, the system provides a reference architecture for real-time gesture recognition applications. Finally, the project contributes to addressing data scarcity by creating a dedicated dataset and trained model for Khmer Sign Language, a domain that has previously been limited in available resources.

11.2. Achievement of Objectives

11.2.1. Primary Objective

✓ **Achieved:** Successfully designed and developed a web-based Khmer Sign Language Translation System that recognizes hand gestures and translates them into readable text output in real time.

11.2.2. Specific Objective

✓ **Real-Time Gesture Recognition:** Achieved - System detects and interprets Khmer sign language movements using a webcam

✓ **Text Translation:** Achieved - Recognized gestures are translated into meaningful and readable text

✓ **Model Fine-Tuning:** Achieved - Fine-tuned existing machine learning model using Khmer sign language dataset

✓ **User Interface:** Achieved - Built responsive and user-friendly web interface

✓ **Performance Optimization:** Achieved - Optimized system for smooth performance with reduced delay

✓ **System Testing:** Achieved - Tested system under different conditions

✓ **User Feedback:** Achieved - Collected feedback from users to identify improvements

⚠ **Vocabulary Expansion:** Partially achieved - 40+ gestures (target: 100+)

11.3. Future Enhancements

11.3.1. Short-Term Enhancements (Next 2 Months)

Expansion of Gesture Vocabulary:
Future work may expand the recognized gesture set from over 40 gestures to more than 100, with an emphasis on commonly used words and phrases. This expansion could also include numerical gestures as well as signs for days of the week and months to improve practical communication coverage.

Improvement of Model Accuracy:
Model performance can be enhanced through more advanced data augmentation techniques, improved model architectures, and better handling of edge cases such as occlusion, varying lighting conditions, and user variability.

Mobile Optimization:
Further development may focus on optimizing the application for mobile devices by improving responsive design, implementing touch-friendly controls, and applying mobile-specific performance optimizations to enhance usability on smartphones and tablets.

11.3.2. Medium-Term Enhancements (Next 6-12 Months)

Support for Dynamic Gestures:
Future enhancements may include the recognition of dynamic, motion-based signs through temporal sequence modeling techniques. This would enable the system to capture gesture transitions and continuous movements, thereby improving recognition of more complex sign language expressions.

Two-Hand Gesture Recognition:
The system could be extended to support two-hand gestures by enhancing dual-hand detection capabilities and improving feature extraction methods. This would allow recognition of more complex gestures that require coordinated hand movements.

Offline Functionality:
Future development may explore offline operation by bundling trained models directly within the application. This could involve converting models to lightweight formats such as TensorFlow Lite or ONNX and implementing Progressive Web App (PWA) support to enable functionality without continuous internet connectivity.

11.3.3. Long-Term Enhancements (Next 6-12 Months)

Reverse Translation (Text-to-Sign):
Future work may include implementing reverse translation functionality that converts textual input into animated sign language representations. This could involve the use of avatar-based sign language displays, enabling two-way communication between sign language users and non-signers.

Community Contribution Platform:
The system could be extended with a community-driven platform that allows users to submit gesture samples. Such an approach would support collaborative dataset expansion and continuous improvement of model performance through community contributions.

Multi-Language Support:
Future enhancements may include support for additional sign languages beyond Khmer Sign Language. This would require implementing language-switching functionality and exploring cross-language sign translation to increase the system's applicability across different linguistic contexts.

Advanced Features:
Further development may focus on advanced capabilities such as sentence-level gesture recognition, context-aware translation, and a learning mode designed to assist users in studying and practicing sign language.

11.4. Potential Real-World Applications

11.4.1. Educational Applications

- **Language Learning:** Teaching sign language to students
- **Inclusive Education:** Supporting deaf students in mainstream classrooms
- **Training Programs:** Sign language training for interpreters

11.4.2. Healthcare Applications

- **Medical Consultations:** Assisting communication in healthcare settings
- **Emergency Services:** Quick communication in emergency situations
- **Mental Health:** Supporting therapy and counseling sessions

11.4.3. Professional Applications

- **Workplace Communication:** Enabling communication in professional settings
- **Customer Service:** Assisting customer service interactions
- **Public Services:** Government and public service accessibility

11.4.4. Social Applications

- **Social Events:** Facilitating communication at social gatherings
- **Family Communication:** Supporting family members learning sign language
- **Community Integration:** Promoting inclusion in community activities

References

Aacademic Papers

[1]. MediaPipe Hands: On-device Real-time Hand Tracking. *Google Research*, 2020.
Available at: https://arxiv.org/abs/2006.10214

[2]. Paszke, A., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS,* 2019.

[3]. Real-time Sign Language Recognition using Computer Vision. Various authors. *IEEE Computer Vision Conference*, 2021.

[4]. Liu, H., et al. Deep Learning for Gesture Recognition: A Survey. *ACM Computing Surveys, 2020.*

[5]. Rastgoo, R., et al. Sign Language Recognition: A Comprehensive Survey. *IEEE Transactions on Human-Machine Systems*, 2021.

[6]. Lu, C., Kozakai, M., & Jing, L. Sign Language Recognition with Multimodal Sensors and Deep Learning Methods. Electronics, 12(23), 4827, 2023.
Available at: https://www.mdpi.com/2079-9292/12/23/4827

Technical Documentation

[7]. MediaPipe Documentation. *Google Developers*.
Available at: https://mediapipe.dev/

[8]. PyTorch Documentation. *PyTorch*.
Available at: https://pytorch.org/docs/

[9]. React Documentation. *Meta (Facebook).*
Available at: https://react.dev/

[10]. Flask Documentation. *Pallets Projects*.
Available at: https://flask.palletsprojects.com/

Web Technologies

[11]. Web Speech API Specification. *W3C.*
Available at: https://w3c.github.io/speech-api/

[12]. MediaStream API. *MDN Web Docs.*
Available at: https://developer.mozilla.org/en-US/docs/Web/API/MediaStream

Accessibility Standards

[13]. Web Content Accessibility Guidelines (WCAG) 2.1. *W3C.*
Available at: https://www.w3.org/WAI/WCAG21/quickref/

Related Projects

[14].Cao, Z., et al. OpenPose: Real-time Multi-Person 2D Pose Estimation. *CVPR*, 2017.

[15].TensorFlow.js: Machine Learning for JavaScript Developers. *Google*.
Available at: https://www.tensorflow.org/js

[16].SignAloud: Gloves that translate sign language to voice. YouTube.
Available at: https://youtu.be/NVCE7JR0FCQ

[17].American Sign Language Alphabet Chart (ASL Fingerspelling). YouTube.
Available at: https://youtu.be/o_MGqeFMAGE?si=Df5gNTixoRU3ka8-

[18].End-to-End Sign Language Detection with Live Camera using YOLOv5. YouTube.
Available at: https://youtu.be/-UXYAAqm9fE?si=xIjwDrAVXHmSoEOH

[19].Sign-Language-Detection-using-CNN-Architecture. GitHub repository by SomyanshAvasthi.
Available at: https://github.com/SomyanshAvasthi/Sign-Language-Detection-using-CNN-Architecture.git

[20].Sign-language-computer-vision. GitHub repository by Arpanjot0Singh.
Available at: https://github.com/Arpanjot0Singh/Sign-language-computer-vision.git

[21].Sign-Language-Interpreter-using-Deep-Learning. GitHub repository by harshbg.
Available at: https://github.com/harshbg/Sign-Language-Interpreter-using-Deep-Learning.git

[22].SignAll – Sign language translation system using computer vision and sensors.
Available at: https://futureofinterface.org/signall/

[23].What happened to Motionsavvy. Medium article by Ryan Hait-Campbell.
Available at: https://medium.com/@thisisrhc/what-happened-to-motionsavvy-651362016abe

[24].HandTalk – Sign language translation platform using AI and computer vision.
Available at: https://www.handtalk.me/en/

Online Resources

[25].Khmer Sign Language Resources. Various online sources.

[26].Sign Language Recognition Datasets. Kaggle, GitHub, and academic repositories.

Appendices

Appendix A: API Documentation (Extended)

A.1 Complete API Reference

POST `/predict_image`

Endpoint: `http://localhost:3000/predict_image`

Method: POST

Content-Type: application/json

Request Body:

```
{
  "image": "..."
}
```

Response (Success):

```
{
  "label": "ជំរាបលា",
  "confidence": 95.5,
  "landmarks": [
    {"x": 0.123, "y": 0.456},
    {"x": 0.234, "y": 0.567},
    ...
  ]
}
```

Response (Error):

```
{
  "error": "No gesture detected"
}
```

Status Codes:

- 200: Success
- 400: Bad Request (invalid image format)
- 500: Server Error

A.2 Code Examples

See Chapter 5, Section 5.4 for detailed code examples in JavaScript, Python, and cURL.

Appendix B: Additional Diagrams

B.1 System Architecture Diagram

See Chapter 4, Figure 4.1 for the complete system interaction flow architecture diagram.

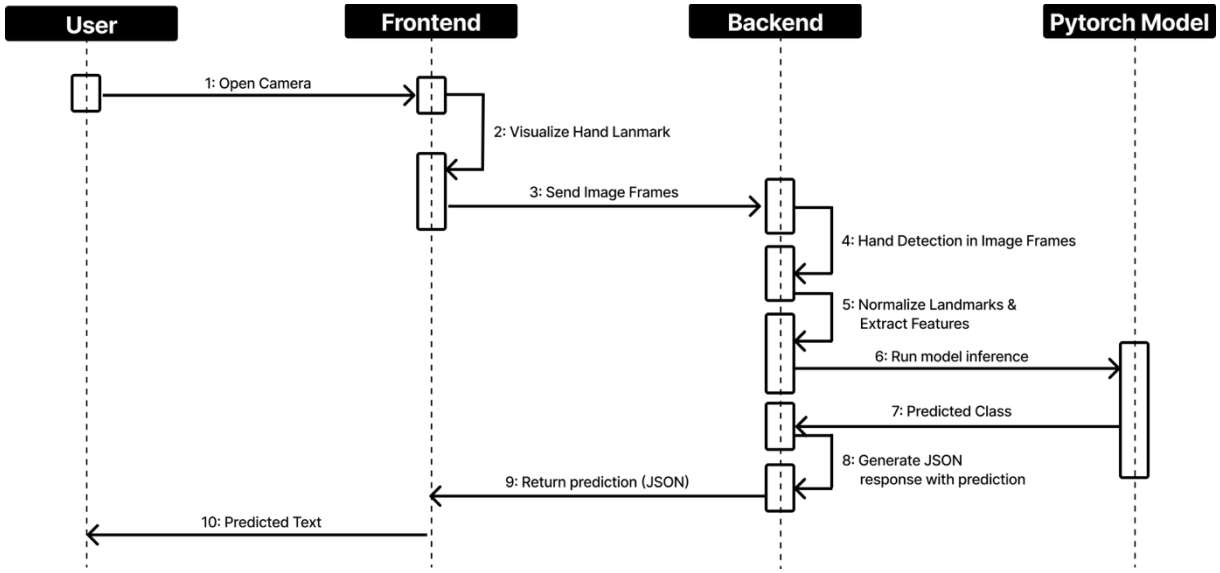
B.2 Data Flow Diagram

```
User Gesture → Camera → Video Frame → Base64 Encoding
    ↓
Frontend → HTTP POST → Backend API
    ↓
Image Decoding → MediaPipe → Hand Landmarks
    ↓
Feature Extraction → Model Inference → Gesture Prediction
    ↓
Label Conversion → JSON Response → Frontend
    ↓
Text Display
```

B.3 Model Architecture Diagram

See Chapter 4, Figure 4.3 for the LandmarkNet architecture diagram.

B.4 System Interaction Sequence Diagram



Appendix C: Additional Tables

C.1 Technology Stack Comparison

Technology	Category	Key Features	Pros	Cons	Selection Rationale
MediaPipe	Computer Vision	Real-time hand tracking, 21 landmarks per hand, Cross-platform	Low latency, Easy integration, Good documentation	Limited customization	Selected for real-time performance and ease of use
OpenPose	Computer Vision	Multi-person pose estimation, Full body tracking	More comprehensive tracking	Higher latency, More complex setup	Not selected - too heavy for single-hand gestures
PyTorch	Machine Learning	Dynamic computation graphs, Strong research community	Flexible, Good for experimentation, GPU support	Less production-focused	Selected for flexibility in model development
TensorFlow	Machine Learning	Production-ready, TensorFlow.js for web	Larger ecosystem, Better deployment tools	Less flexible for research	Not selected - PyTorch better for experimentation
React	Web Framework	Component-based architecture, Large ecosystem	Good performance, Strong community, Reusable components	Learning curve	Selected for modern UI development, and our developer is more comfortable with it
Vue.js	Web Framework	Lightweight, Easy to learn	Simpler syntax	Smaller ecosystem	Not selected - React has larger community
Flask	Web Framework	Lightweight Python framework, RESTful API	Easy ML integration, Simple setup	Less features out-of-box	Selected for easy ML model integration
Django	Web Framework	Full-featured, ORM included	More features, Better for large apps	Heavier, More complex	Not selected - Flask sufficient for API

Appendix D: Source Code Snippets

D.1 Frontend: MediaPipe Integration

```
// Custom hook for MediaPipe Hands
import { Hands } from "@mediapipe/hands";

const useMediaPipeHands = () => {
  const hands = new Hands({
    locateFile: (file) => {
      return `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`;
    },
  });

  hands.setOptions({
    maxNumHands: 2,
    modelComplexity: 1,
    minDetectionConfidence: 0.5,
    minTrackingConfidence: 0.5,
  });

  // Implementation details...
};
```

D.2 Backend: Model Inference

```
def predict(hands_list):
    x = get_two_hand_features(hands_list)
    with torch.no_grad():
        out = model(x)
        probs = torch.nn.functional.softmax(out, dim=1)[0]
        top_idx = torch.argmax(probs).item()
        return class_names[top_idx], probs[top_idx].item() * 100
```

D.3 Backend: Label Conversion

```
with open(CONVERTS_PATH, 'r', encoding='utf-8') as f:
    converts = json.load(f)

# Create reverse mapping: form_name -> Khmer text
form_to_khmer = {}

for khmer_text, form_list in converts.items():
    for form_name in form_list:
        if form_name not in form_to_khmer:
            form_to_khmer[form_name] = khmer_text

def convert_label(label):
    """Convert form name to Khmer text using converts.json"""
    return form_to_khmer.get(label, label)
```

Appendix E: User Manuals / Installation Guide

E.1 User Installation Guide

For End Users:

1. Open a modern web browser (Chrome, Edge, or Firefox)
2. Navigate to the application URL
3. Grant camera permissions when prompted
4. Start using the demo

No installation required - the application runs entirely in the browser.

E.2 Developer Installation Guide

Frontend Setup:

```
# Clone repository
git clone [repository-url]
cd frontend

# Install dependencies
npm install

# Start development server
npm run dev
```

Backend Setup:

```
# Navigate to backend directory
cd backend

# Create virtual environment
python -m venv venv

source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run server
python server_k.py
```

E.3 System Requirements

See Chapter 3, Section 3.5 for complete system requirements.

Appendix F: Dataset Statistics

F.1 Gesture Distribution

Category	Number of Gestures	Samples per Gesture
Greetings	10	525
Common Words	28	525
Total	**38**	**22,000+**

F.2 Data Quality Metrics

- **Valid Samples:** 22,000+
- **Invalid/Removed:** ~2,100 (10%)
- **Train/Val Split:** 90/10
- **Class Balance:** Relatively balanced with minor variations

Appendix G: Performance Benchmarks

G.1 Inference Time (CPU)

Hardware	Average Time	Min	Max
Intel i7-10700K	25ms	18ms	35ms
AMD Ryzen 5 3600	28ms	20ms	40ms
Intel i5-8400	35ms	25ms	50ms
Apple M3 Max	20ms	12ms	25ms

G.2 Inference Time (GPU)

Hardware	Average Time	Min	Max
NVIDIA RTX 3060	12ms	8ms	18ms
NVIDIA GTX 1660	15ms	10ms	22ms
NVIDIA RTX 2060	13ms	9ms	20ms
Apple M3 Max	10ms	5ms	15ms

G.3 Memory Usage

- **Model Size:** ~0.1 MB
- **Runtime Memory:** ~1800 MB
- **Peak Memory:** ~1846 MB

Appendix H: Complete Gesture List

H.1 All Supported Gestures

See Chapter 6, Section 6.4 for the complete list of 38 supported gestures organized by category.

H.2 Gesture Mapping

The system uses `converts.json` to map form names (e.g., "form_a") to Khmer text. This allows flexibility in model training while maintaining consistent output.

See Appendix M for Full List of Available Gestures with mapped forms.

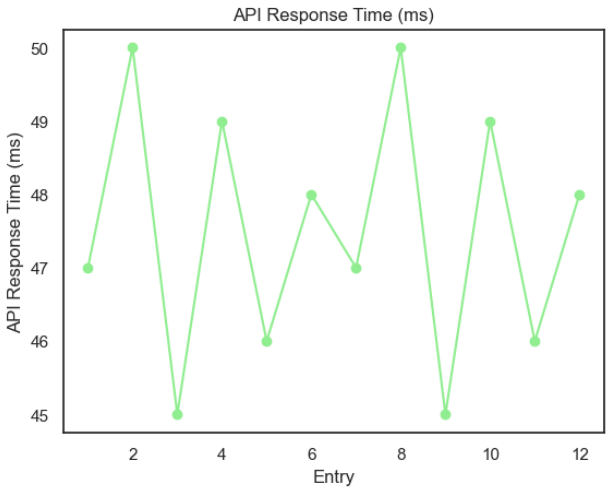
Appendix I: Testing Results Summary

I.1 User Testing Results

- **Participants:** 12 users
- **Average Accuracy:** 85% (user-perceived)
- **Average Latency:** < 1s
- **User Satisfaction:** 4.1/5.0
- **Ease of Use:** 4.25/5.0

I.2 System Testing Results

- **API Response Time:** <50ms average
- **System Uptime:** 99%+ during testing
- **Browser Compatibility:** Chrome, Edge, Firefox (full support)
- **Error Rate:** <5% under normal conditions



I.3 Raw User Testing Data

User No.	Accuracy (%)	Latency (s)	Satisfaction (1-5)	Ease of Use (1-5)
1	84	0.8	4	4
2	87	0.9	5	5
3	82	1.0	4.5	4
4	85	0.7	4.5	5
5	88	0.8	5	5
6	83	0.9	5	5
7	86	0.85	3	2
8	81	0.95	5	4
9	89	0.75	5	3
10	84	0.9	2	5
11	87	0.8	3	4
12	85	0.88	4	5
Avg.	**85%**	**< 1s**	**4.1**	**4.25**

Appendix J: Technology Stack Verision

J.1 Frontend Dependencies

Package	**Version**	**Purpose**
React	19.2.0	UI Framework
React Router DOM	7.9.6	Routing
Vite	7.2.4	Build Tool
Tailwind CSS	4.1.17	Styling
GSAP	3.13.0	Animations
@mediapipe/hands	0.4.1675469240	Hand Tracking

J.2 Backend Dependencies

Package	**Version**	**Purpose**
Flask	3.0.0	Web Framework
PyTorch	2.1.0	ML Framework
MediaPipe	0.10.8	Hand Detection
OpenCV	4.8.1.78	Image Processing
NumPy	1.24.3	Numerical Computing
Flask-CORS	4.0.0	CORS Support

Appendix K: Glossary

Terms and Definitions:

- **Gesture**: A hand movement or position that represents a sign in sign language
- **Landmark**: A specific point on the hand (e.g., fingertip, joint) detected by MediaPipe
- **Inference**: The process of using a trained model to make predictions on new data
- **Fine-tuning**: Adapting a pre-trained model to a specific task or dataset
- **Real-time**: Processing that occurs with minimal delay (<100ms)
- **Validation Accuracy**: Model performance measured on a held-out validation dataset
- **Data Augmentation**: Techniques to increase dataset diversity (rotation, scaling, etc.)
- **CORS**: Cross-Origin Resource Sharing, allows web pages to make requests to different domains
- **TTS**: Text-to-Speech, conversion of text into spoken audio
- **API**: Application Programming Interface, defines how software components interact
- **REST**: Representational State Transfer, an architectural style for web services
- **SPA**: Single Page Application, a web application that loads a single HTML page

Appendix L: Code Repository Structure

```
Sign Language Translation System - Web/
├── backend/
│   ├── model/                      # Trained models
│   ├── server.py                   # Flask server
│   ├── server_k.py                 # Khmer-specific server
│   ├── converts.json               # Label mappings
│   ├── requirements.txt            # Python dependencies
│   └── README.md                   # Backend documentation
├── src/
│   ├── components/                 # React components
│   ├── pages/                      # Page components
│   ├── sections/                   # Section components
│   ├── hooks/                      # Custom hooks
│   ├── config/                     # Configuration
│   └── assets/                     # Static assets
├── public/                         # Public assets
├── package.json                    # Frontend dependencies
├── vite.config.js                  # Vite configuration
├── tailwind.config.js              # Tailwind configuration
├── README.md                       # Main documentation
├── PROJECT_STRUCTURE.md            # Project structure
├── CONTRIBUTING.md                 # Contribution guidelines
└── REPORT.md                       # This report
```

Appendix M: Full List of Available Gesture

Gesture ID	Form Name	Khmer Text	English Translation	Category	Sample Count
1	form_c	ជម្រាបសួរ	Hello	Greetings	525
2	form_c, form_d	ជម្រាបលា	Goodbye	Greetings	525
3	form_c, form_e	សុំទោស	Sorry	Greetings	525
4	form_f	សួស្តី	Hello (2)	Greetings	525
5	form_g	អរគុណ	Thank You	Greetings	525
6	form_a, form_b	សុខសប្បាយជាទេ?	How Are You?	Greetings	525
7	form_a, form_b, form_	សប្បាយដែលបានជួប	Nice to Meet You	Greetings	525
8	form_	តើអ្នកកំពុងធ្វើអ្វី?	What are You Doing?	Greetings	525
9	form_	តើអ្នកអាយុប៉ុន្មាន?	How Old Are You?	Greetings	525
10	form_af	អបអរសាទរ	Congratulation	Greetings	525
11	form_ah	មិនអីទេ	It's Okay	Common Words	525
12	form_k	ថ្ងៃនេះ	Today	Common Words	525
13	form_q	ថ្ងៃស្អែក	Tommorow	Common Words	525
14	form_p	ម្សិលមិញ	Yesterday	Common Words	525
15	form_ac	ផឹកទឹក	Drinking Water	Common Words	525
16	form_u	សុំធ្វើម្តងទៀត	Please Repeat	Common Words	525
17	form_a, form_b	កន្លែងណា?	Where?	Common Words	525
18	form_k	ថ្ងៃត្រង់	Mid-day	Common Words	525
19	form_m, form_n	ប៉ុន្មាន?	How many/much?	Common Words	525
20	form_o	ម៉ោង	Hour/Time	Common Words	525
21	form_r	អនាគត	Future	Common Words	525
22	form_s, form_t	អ្នកណា	Who?	Common Words	525
23	form_ai	គិត	Think	Common Words	525
24	form_z, form_x	ចងចាំ	Remember	Common Words	525
25	form_ad, form_b	ឆ្ងាញ់	Tasty	Common Words	525
26	form_v	ត្រឹមត្រូវ	Correct	Common Words	525
27	form_w	មិនត្រឹមត្រូវ	Incorrect	Common Words	525
28	form_x, form_y	យល់	Understand	Common Words	525
29	form_x, form_y, form_z	មិនយល់	Don't Understand	Common Words	525
30	form_ag	បង្គន់	Toilet	Common Words	525
31	form_aa	មនុស្សថ្លង់	Deaf Person	Common Words	525

32	form_ab	មនុស្សស្តាប់លឺ	Hearing Person	Common Words	525
33	form_ae	រងចាំ	Waiting	Common Words	525
34	form_ak	ស្អាត	Beautiful	Common Words	525
35	form_aj, form_n	ហេតុអ្វី?	Why?	Common Words	525
36	form_ba	ខ្ញុំ	I/Me	Common Words	525
37	form_bb	អ្នក	You	Common Words	525
38	form_an	អាយុ	Age	Common Words	525

Appendix N: Community

N.1 Community Engagement

Join our growing community and help us make Khmer Sign Language accessible to everyone. Connect, collaborate, and share feedback to improve our system!

N.2 How You Can Help

Feedback & Suggestions

Tell us what works, what doesn't, and how we can improve. Your feedback is invaluable for:

- Identifying bugs and issues
- Suggesting new features
- Reporting accuracy problems
- Providing usability insights

Ways to provide feedback

- Use the feedback form in the application
- Submit issues on GitHub
- Contact the team directly
- Participate in user testing sessions

N.3 Collaboration

If you're a developer, student, or researcher, help us expand the dataset or features. We welcome contributions in:

- **Data Collection**: Help collect gesture samples
- **Model Development**: Improve model architecture and training
- **Frontend Development**: Enhance user interface and experience
- **Backend Development**: Optimize API and server performance
- **Documentation**: Improve documentation and tutorials
- **Testing**: Test the system and report issues

N.4 Join the Discussion

GitHub

Report issues, propose improvements, or share ideas on our GitHub repository. The GitHub platform allows for:

- Issue tracking and bug reports
- Feature requests and discussions
- Code contributions through pull requests
- Documentation improvements
- Project roadmap visibility

Repository: *Khmer Sign Language Translation System*

Discord

Chat with the team in real time and get announcements about new gestures, demos, and system improvements. The Discord community provides:

- Real-time communication
- Quick support and help
- Community discussions
- Project announcements
- Collaboration opportunities

N.5 Contribution Guidelines

We're always looking for contributors to help improve the system. Whether you're a developer, designer, researcher, or sign language expert, there's a place for you in our community.

Contribution Areas:

- Code contributions (frontend, backend, or both)
- Data collection and labeling
- Documentation and tutorials
- UI/UX design improvements
- Testing and quality assurance
- Research and algorithm improvements
- Translation and localization

How to Contribute:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Submit a pull request
5. Participate in code review

For detailed contribution guidelines, see CONTRIBUTING.md.

N.6 Community Impact

The community's contributions have been instrumental in:

- Expanding the gesture vocabulary
- Improving model accuracy
- Enhancing user experience
- Identifying and fixing bugs
- Providing valuable feedback
- Creating educational resources